VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Final Section: Final Interview Preparation

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

Final Interview Preparation: How to Use This Section 4
- How the Section Is Organised 4
- How to Answer a Technical Question 5
- The Most Frequently Asked Topics 5

Basic Questions 7
- Language and Data Types 7
- Arrays 7
- Tasks and Functions 8
- Classes and OOP 8
- Randomization 9
- Verification Concepts 10
- Inter-Process Communication 10
- Interfaces and Testbench 11
- Coverage, Assertions and Other Topics 11

Intermediate Questions 13
- Procedural Code and Arrays 13
- Tasks and Functions 14
- Classes 15
- Randomization and Constraints 16
- Inter-Process Communication 17
- Interfaces and the Testbench 18
- Coverage and Assertions 19
- Packages, DPI and UVM 20

Advanced Questions 21
- Scheduling and Timing 21
- The Constraint Solver 22
- Classes and Memory 23
- Process Control 24
- Coverage Semantics 25
- Assertion Semantics 25

Output-Prediction Questions 27
- Arrays and Strings 27
- Tasks and Functions 30
- Classes 31
- Threads and Communication 35
- Assignments and Randomization 38

Code-Debugging Questions 41

Find-the-Error Questions 47

Write-the-Code Questions 51
- Arrays and Basic Logic 51
- Classes 55
- Threads and Communication 57

Constraint-Writing Questions 61
Single Variables 61
Relations Between Variables 63
Arrays 64

OOP, IPC and Testbench Architecture Questions 67
OOP-Based Questions 67
IPC-Based Questions 69
Layered Testbench Architecture Questions 71

Industry Scenario Questions 74
Debugging Scenarios 74
Verification-Planning Scenarios 76

Frequently Asked Fresher Questions 79
Last-Day Revision Checklist 81

Coding Exercises with Solutions 83
Exercise 1: A Parameterised FIFO Class 83
Exercise 2: A Synchronous FIFO and a Self-Checking Testbench 84
Exercise 3: Transaction Copy and Compare 86
Exercise 4: Polymorphism with an Abstract Base Class 87
Exercise 5: Constraints with a Procedural Self-Check 88
Exercise 6: A Three-Stage Pipeline with Mailboxes 90
Exercise 7: Three Users, Two Keys 91
Exercise 8: Killing Slow Threads with Process Handles 91
Exercise 9: A Counter with Assertions and Coverage 92
Exercise 10: A Compact Layered Testbench for the ALU 94

Final Interview Preparation: How to Use This Section

This section is the question bank for the whole course. It contains more than 300 numbered questions with answers, grouped by the kind of question an interviewer asks, not by module. Use it after you have studied Modules 1 to 11.

Cross-references. "M5" means Module 5. The modules are: M1 data types, M2 arrays, M3 tasks and functions, M4 classes and OOP, M5 randomization and constraints, M6 transactions and verification planning, M7 inter-process communication, M8 interfaces, M9 layered testbench, M10 coverage and assertions, M11 packages, advanced topics and UVM.

Naming. The examples use the course names: the DUT `alu` with the opcode type `alu_op_e`, the interface `alu_if` with clocking blocks `drv_cb` and `mon_cb`, the transaction `alu_txn`, the components `alu_generator`, `alu_driver`, `alu_monitor`, `alu_scoreboard`, `alu_env`, `alu_test`, the mailboxes `gen2drv_mbx` and `mon2scb_mbx`, and the FIFO `sync_fifo`.

> **Note:** No simulator was available when these notes were written. Every predicted output was derived by hand from the IEEE 1800-2017 rules. Output-prediction questions were chosen so that the result is fully determined by the language (no dependence on random seeds or on the order in which a simulator runs threads that wake at the same time). Run the code yourself; if your simulator prints something different, find out why.

How the Section Is Organised

Part	Questions	Content
Basic questions	Q1 to Q55	One- or two-sentence answers; the first ten minutes of an interview
Intermediate questions	Q56 to Q110	"Explain the difference", "what happens if"
Advanced questions	Q111 to Q146	Scheduling regions, solver behaviour, polymorphism details, sampling
Output prediction	Q147 to Q170	Short programs; you state the exact output
Code debugging	Q171 to Q186	Code that compiles but misbehaves
Find the error	Q187 to Q202	Code that does not compile
Write the code	Q203 to Q219	Small programs written on a whiteboard or shared editor
Constraint writing	Q220 to Q243	The most common coding questions in DV interviews
OOP questions	Q244 to Q259	
IPC questions	Q260 to Q275	
Layered testbench questions	Q276 to Q291	
Industry scenarios	Q292 to Q308	"The test hangs. What do you do?"
Fresher questions	Q309 to Q326	Including "explain your project"
Coding exercises	10 exercises	Larger programs with complete solutions

How to Answer a Technical Question

A good answer has a fixed shape. Practise it until it is automatic.

1. **Definition.** One sentence saying what the thing is, using the correct term.
2. **Why it exists.** One sentence about the problem it solves.
3. **Example.** A two- or three-line code fragment, or a concrete use in a testbench.
4. **Pitfall or comparison.** One thing people get wrong, or how it differs from the nearest related concept.

Example for "What is a virtual interface?":

> **How to say it in an interview:** "A virtual interface is a handle, a variable that refers to an actual interface instance. It is needed because classes are dynamic objects and cannot contain an interface instance or connect to module ports, so the class holds a handle and the testbench top assigns the real interface to it. For example the driver has `virtual alu_if vif;` and drives `vif.drv_cb.a <= txn.a;`. The common mistake is to use it before it is assigned, which gives a null-handle error at run time."

Rules that apply to every answer:

- Answer the question that was asked first, then add detail. Do not start with history.
- If you do not know, say what you do know and how you would find out. Guessing confidently is worse than "I have not used that; my understanding is ...".
- When writing code, say what you are doing while you write. State assumptions ("I assume the reset is active low").
- When asked to predict output, go line by line and say the state after each line. Interviewers give credit for the method even when the final answer is wrong.
- When asked to debug, do not guess the fix. Say how you would narrow it down: reproduce with the same seed, find the first wrong thing in the log or waveform, form a hypothesis, test it.

The Most Frequently Asked Topics

If you have only one evening, make sure you can answer every line below without notes. The number in brackets is where the full answer is.

Topic	Question	Where
Data types	`logic` vs `bit` vs `reg` vs `wire`; two-state vs four-state	M1, Q3
Data types	`==` vs `===` vs `==?`	M1
Arrays	Static vs dynamic vs associative vs queue: when to use each	Q57
Arrays	Queue methods; array locator methods	Q9, Q61
Procedural	Task vs function; `automatic` vs `static`; `ref`	Q13, Q15, Q65
Procedural	Blocking vs nonblocking assignment	Q56
OOP	Class vs object vs handle; shallow vs deep copy	Q18, Q69
OOP	Inheritance, polymorphism, virtual methods, `$cast`	Q26, Q72, Q73
OOP	Abstract class, pure virtual method	Q28

Topic	Question	Where
Randomization	`rand` vs `randc`; `randomize()` return value	Q29, Q30
Randomization	`inside`, `dist`, implication, `solve before`, `soft`, `unique`	Q78 to Q82
Randomization	`constraint_mode()` vs `rand_mode()`; pre/post randomize	Q35, Q36
Randomization	Writing constraints for arrays	Q220 to Q243
IPC	`fork` ... `join` vs `join_any` vs `join_none`; `disable fork`; `wait fork`	Q44, Q89
IPC	Mailbox vs semaphore vs event; `@e` vs `wait(e.triggered)`	Q87, Q88
Interfaces	Interface, modport, clocking block, virtual interface	Q45 to Q48
Testbench	Role of generator, driver, monitor, scoreboard, environment, test	Q49
Testbench	How data flows and how the test ends	Q276, Q282
Coverage	Code vs functional coverage; bins; cross; ignore vs illegal bins	Q39, Q99
Assertions	Immediate vs concurrent; `|->` vs `|=>`; `disable iff`; `$past`	Q50, Q51
Scheduling	Event regions; why clocking blocks remove races	Q111, Q113
UVM	What it is and how it maps to plain SystemVerilog	Q55, Q110

Basic Questions

These are the questions asked in the first minutes of an interview to check that you know the vocabulary. The answers are deliberately short: say this much, then stop and let the interviewer ask for more.

Language and Data Types

Q1. What is functional verification and why is it needed? Functional verification is the process of proving, by simulation and other methods before manufacturing, that the RTL design behaves as its specification says. It is needed because a bug found in silicon costs a new mask set and months of delay, and because the number of possible input scenarios is far too large to be checked by inspection.

Q2. What is the difference between the DUT and the testbench? The DUT (design under test) is the synthesizable RTL that will become hardware. The testbench is simulation-only code that generates stimulus for the DUT, observes its outputs and checks them. The testbench is never synthesized, so it may use the whole language.

Q3. Which built-in types are two-state and which are four-state, and where do you use each? Two-state (0, 1): `bit`, `byte`, `shortint`, `int`, `longint`. Four-state (0, 1, X, Z): `logic`, `reg`, `integer`, `time` and all nets such as `wire`. Four-state types are used for anything connected to the DUT so that X and Z stay visible; two-state types are used for testbench data such as random fields, counters and indices because they are faster and never need X or Z. `reg` and `logic` are the same type; `wire` is a net, required when a signal has several drivers.

Q4. Why can a `logic` signal not be used for a bidirectional bus? `logic` declares a variable, and a variable may have only one driver. A bidirectional bus has several drivers that take turns, so it needs a net such as `wire`, whose value is resolved from all its drivers.

Arrays

Q5. What is the difference between a fixed-size array and a dynamic array? The size of a fixed-size (static) array is given in the declaration and never changes: `int a [8];`. The size of a dynamic array is set at run time with `new[]` and can be changed later: `int d []; d = new[8];`.

Q6. What is a queue? A variable-size ordered list declared with `[$]`, for example `int q [$];`. Elements can be added and removed at both ends in constant time and anywhere in the middle. It grows and shrinks automatically.

Q7. What is an associative array and when is it used? An array whose index can be of any type (integer, string, enum, class handle) and which allocates storage only for the entries that are written: `int mem [bit [31:0]];`. It is used for sparse data such as a memory model with a large address space, and as a lookup table keyed by a string or an ID.

Q8. How do you allocate a dynamic array and how do you resize it while keeping the contents? `d = new[10];` allocates ten elements with default values. `d = new[20](d);` allocates twenty elements and copies the old ten into the first ten positions. `d.delete();` empties the array.

Q9. Name the queue methods. `size()`, `insert(index, item)`, `delete()` (all elements) or `delete(index)`, `push_back(item)`, `push_front(item)`, `pop_back()`, `pop_front()`. The array manipulation methods (`find`, `sort`, `sum` and so on) also work on queues.

Q10. What do `exists()`, `num()` and `delete()` do on an associative array? `exists(key)` returns 1 if an entry with that key is present. `num()` (or `size()`) returns the number of entries. `delete(key)` removes one entry and `delete()` removes all entries.

Q11. What is `foreach`? A loop that visits every element of an array and declares the loop index automatically: `foreach (a[i]) a[i] = i;`. It works on all array types, in index order, and for an associative array it visits only the entries that exist.

Q12. What do the array locator methods return? A queue. `find()`, `find_first()`, `find_last()`, `min()`, `max()` and `unique()` return a queue of elements; `find_index()`, `find_first_index()`, `find_last_index()` and `unique_index()` return a queue of indices. Even the methods that find at most one item return a queue (with zero or one element).

Tasks and Functions

Q13. What is the difference between a function and a task? A function executes in zero simulation time, cannot contain delays or event controls, and can return a value, so it can be used inside an expression. A task may consume simulation time, has no return value (it passes results back through `output`, `inout` or `ref` arguments), and is called as a statement. A task can call functions and tasks; a function can call only functions.

Q14. What is a void function? A function declared with return type `void`. It returns nothing and is called as a statement, like a task, but it is still not allowed to consume time. To discard the return value of a non-void function, write `void'(f());`.

Q15. What is the difference between `automatic` and `static` lifetime? A static task or function has one copy of its arguments and local variables, shared by every call. An automatic task or function allocates new storage for each call, so it is re-entrant and can be recursive. Tasks and functions in modules, interfaces and packages are static by default; methods of a class are always automatic.

Q16. What is the difference between passing an argument by value and by reference? By value (`input`, `output`, `inout`) the argument is copied into the routine at the call and, for `output` and `inout`, copied back when the routine returns. By reference (`ref`) no copy is made: the routine works directly on the caller's variable, so changes are visible immediately and large arrays are not copied.

Q17. What are default argument values? Values written in the declaration that are used when the caller omits the argument: `task send(int n = 1, bit verbose = 0);`. The call `send();` uses 1 and 0. Arguments can also be passed by name: `send(.verbose(1));`.

Classes and OOP

Q18. What is the difference between a class, an object and a handle? A class is a type: a description of properties (data) and methods (tasks and functions). An object is an instance of the class, created at run time with `new`. A handle is a variable that refers to an object. Declaring `alu_txn t;` creates only a handle with the value `null`; `t = new();` creates the object.

Q19. What does `new()` do? It allocates memory for an object, initialises the properties to their default or declared values, runs the constructor function `new` of the class (after the constructors of its base classes), and returns a handle to the object.

Q20. What is a null handle and what happens if you use one? A handle that does not refer to any object; `null` is the default value of every class handle. Accessing a non-static property or calling a non-static method through a null handle is a fatal run-time error (null object access).

Q21. What is `this`? A predefined handle, available inside non-static class methods, that refers to the object on which the method was called. It is most often used when an argument has the same name as a property: `this.addr = addr;`.

Q22. What is `super`? A keyword used inside a derived class to refer to the members of its base class, for example `super.new()` to call the base constructor or `super.print()` to call the base version of an overridden method.

Q23. What is a static property? A class property declared with `static`. There is one copy shared by all objects of the class, it exists even when no object has been created, and it can be accessed with the class name: `alu_txn::count`. A typical use is counting objects or giving each one a unique ID.

Q24. What is the difference between `local` and `protected`? A `local` member can be accessed only by the methods of the class that declares it. A `protected` member can also be accessed by the methods of derived classes. Neither can be accessed from outside. Members without a qualifier are public.

Q25. What is inheritance? Creating a new class from an existing one with `extends`. The derived class gets all properties and methods of the base class and can add new ones or override existing ones. SystemVerilog allows only one base class per class.

Q26. What is polymorphism? The ability of one base-class handle to refer to objects of different derived classes, with a call to a virtual method executing the version that belongs to the actual object. It lets testbench code be written once for the base type and work for every extension.

Q27. What is a virtual method? A method declared with `virtual`. When it is called through a handle, the implementation is selected at run time from the type of the object, not the type of the handle. Without `virtual`, the type of the handle decides.

Q28. What is an abstract class and a pure virtual method? An abstract class is declared `virtual class`. It cannot be constructed; it exists to be extended. A pure virtual method (`pure virtual function ...;`) is a prototype with no body inside an abstract class; every non-abstract derived class must implement it.

Randomization

Q29. What is the difference between `rand` and `randc`? A `rand` variable gets a value chosen uniformly from its legal values on every call, so values may repeat. A `randc` variable is random-cyclic: it does not repeat a value until every legal value has been used once, then a new cycle starts.

Q30. What does `randomize()` return? 1 if the solver found values that satisfy all active constraints and assigned them, 0 if it failed. On failure the variables keep their old values. The result must always be checked, for example `if (!t.randomize()) $fatal(1, "randomize failed");`.

Q31. What is a constraint block? A named block inside a class that lists Boolean expressions which the random variables must satisfy: `constraint addr_c { addr inside {[0:99]}; addr % 4 == 0; }`. All expressions in all active blocks must be true at the same time.

Q32. What does the `inside` operator do? It tests whether a value is a member of a set of values and ranges: `x inside {1, 3, [10:20]}`. In a constraint it restricts a variable to that set, each member being equally likely. `!(x inside {...})` excludes the set.

Q33. In a `dist` constraint, what is the difference between `:=` and `:/`? `:=` gives the weight to every value in the range; `:/` divides the weight among the values in the range. In `x dist {[1:4] := 40, 5 := 60}` each of 1 to 4 has weight 40 (total 220); in `x dist {[1:4] :/ 40, 5 := 60}` each of 1 to 4 has weight 10 (total 100).

Q34. What is a soft constraint? A constraint marked `soft`, which the solver satisfies if it can and silently drops if it conflicts with a hard (normal) constraint or a higher-priority soft constraint. It is used for default values that a test may override: `constraint len_c { soft len == 4; }`.

Q35. What are `pre_randomize()` and `post_randomize()`? Built-in functions that `randomize()` calls automatically, before solving and after a successful solve. `pre_randomize()` is used to set up state or switch constraints; `post_randomize()` is used to compute dependent fields such as a checksum. They are functions, so they cannot consume time.

Q36. What is the difference between `constraint_mode()` and `rand_mode()`? `constraint_mode(0)` switches a constraint block off, so the solver ignores it. `rand_mode(0)` switches the randomness of a variable off, so the solver treats it as a fixed state value (and still checks it against the constraints). Both take 1 to switch back on, and both can be applied to one item (`t.addr_c.constraint_mode(0)`, `t.addr.rand_mode(0)`) or to the whole object.

Verification Concepts

Q37. What is a transaction? A high-level description of one operation on an interface, held in a class object: for the ALU, one set of `a`, `b`, `op` and the resulting `result`. Testbench components pass transactions to each other; only the driver and monitor deal with individual signals and clock edges.

Q38. What is the difference between directed and constrained-random testing? In a directed test the engineer writes each stimulus value by hand to hit one scenario. In constrained-random testing the testbench generates many random stimuli limited to legal values by constraints, a scoreboard checks them automatically, and functional coverage records which scenarios occurred. Random testing finds cases nobody thought of; directed tests fill the last coverage holes.

Q39. What is the difference between code coverage and functional coverage? Code coverage is collected automatically by the simulator and reports which parts of the RTL were exercised (lines, branches, conditions, toggles, FSM states). Functional coverage is written by the verification engineer with covergroups and reports which features and scenarios of the specification were exercised. 100% code coverage does not show that a required feature was implemented; 100% functional coverage does not show that all code was exercised. Both are needed.

Inter-Process Communication

Q40. What is a mailbox? A built-in class that works as a FIFO between processes. One process puts items in, another gets them out, and the mailbox synchronises them: `get()` blocks while the mailbox is empty. It is the standard way to pass transactions from the generator to the driver and from the monitor to the scoreboard.

Q41. What are the mailbox methods? `new(bound)` creates it (no argument or 0 means unbounded). `put()` adds an item, blocking if a bounded mailbox is full. `get()` removes the first item, blocking if empty. `peek()` copies the first item without removing it, blocking if empty. `try_put()`, `try_get()` and `try_peek()` never block and return a status. `num()` returns the number of items.

Q42. What is a semaphore? A built-in class that holds a number of keys. A process must `get()` a key before using a shared resource and `put()` it back afterwards; `get()` blocks while not enough keys are available. With one key it gives mutual exclusion, for example two sequences sharing one bus driver.

Q43. What is an event and how is it used? A synchronisation object with no data. One process triggers it with `-> e;` and other processes wait for it with `@e;` or `wait (e.triggered);`. It is used to signal that something happened, for example that the generator has finished.

Q44. What is the difference between `fork`...`join`, `join_any` and `join_none`? All three start the statements inside as parallel threads. `join` blocks the parent until all threads finish. `join_any` blocks until the first thread finishes; the others keep running. `join_none` does not block at all; the threads start when the parent next blocks or finishes.

Interfaces and Testbench

Q45. What is an interface? A construct that bundles the signals connecting blocks into one named unit that is instantiated once and passed through ports as a single connection. It can also contain modports, clocking blocks, assertions, tasks and functions. It removes repeated port lists and is the connection point between the DUT and a class-based testbench.

Q46. What is a modport? A named view of an interface that declares the direction of each signal for one kind of user: `modport DUT (input a, b, op, output result);`. Different modports give the DUT, the driver and the monitor different directions, so the compiler can report a component that drives a signal it should only read.

Q47. What is a virtual interface? A variable (handle) that refers to an interface instance: `virtual alu_if vif;`. Classes cannot contain interface instances, so a class holds a virtual interface and accesses the DUT signals through it. The testbench top assigns the real interface instance to the handle.

Q48. What is a clocking block? A block inside an interface that groups signals synchronous to one clock and defines when the testbench samples inputs and drives outputs relative to that clock edge (the skews). It removes races between the testbench and the DUT and makes the driver and monitor code cycle-based: `@(vif.drv_cb); vif.drv_cb.a <= 8'h10;`.

Q49. What do the generator, driver, monitor, scoreboard, environment and test do? The generator creates and randomizes transactions. The driver takes transactions and converts them into pin-level activity on the interface. The monitor watches the interface and converts pin activity back into transactions. The scoreboard computes the expected result and compares it with what the monitor observed. The environment creates and connects these components. The test configures the environment (number of transactions, constraints) and starts it.

Coverage, Assertions and Other Topics

Q50. What is the difference between an immediate and a concurrent assertion? An immediate assertion is a procedural statement, `assert (expr) else $error(...);`, checked once at the moment it is executed, like an

`if`. A concurrent assertion, `assert property (@(posedge clk) ...);`, is evaluated on every clock tick using sampled values and can describe behaviour that spans several cycles.

Q51. What is the difference between `|->` and `|=>`? Both are implication operators: if the antecedent (left side) matches, the consequent (right side) must hold. With overlapped implication `|->` the consequent starts in the same clock tick in which the antecedent ends. With non-overlapped implication `|=>` it starts one tick later. `a |=> b` is the same as `a |-> ##1 b`.

Q52. What are a covergroup, a coverpoint, bins and a cross? A covergroup is the container for a coverage model and defines when it is sampled. A coverpoint names a variable or expression to be covered. Bins divide the values of a coverpoint into the cases that are counted. A cross records combinations of the bins of two or more coverpoints.

Q53. What is a package? A named scope that holds declarations to be shared: typedefs, parameters, functions, tasks and classes. Other code uses them with `import pkg::*;` or `pkg::name`. A package cannot contain modules, interfaces or wires.

Q54. What is the difference between `$display`, `$write`, `$strobe` and `$monitor`? `$display` prints immediately and adds a newline. `$write` is the same without the newline. `$strobe` prints once at the end of the current time step, after nonblocking assignments have taken effect. `$monitor` is started once and then prints at the end of every time step in which one of its arguments changed.

Q55. What is UVM? The Universal Verification Methodology: a standard class library (IEEE 1800.2) written in SystemVerilog, plus rules for using it. It provides base classes for the components of a layered testbench (driver, monitor, agent, scoreboard, environment, test), a factory for replacing classes, a configuration database, phases that order the simulation, and sequences for stimulus. It standardises the structure taught in M9 so that testbenches are reusable across projects and companies.

Intermediate Questions

These questions test whether you understand behaviour and can compare related features. Most answers include a small code fragment, because that is how you should answer them in an interview.

Procedural Code and Arrays

Q56. What is the difference between blocking and nonblocking assignment? A blocking assignment (`=`) evaluates the right-hand side and updates the left-hand side immediately, before the next statement runs. A nonblocking assignment (`<=`) evaluates the right-hand side immediately but schedules the update for the NBA region at the end of the time step, so every nonblocking assignment in the same time step sees the old values.

```
a = b;  b = a;      // both end up with the old value of b
a <= b; b <= a;     // the values are swapped
```

Use nonblocking assignments for sequential logic (`always_ff`) and blocking assignments for combinational logic (`always_comb`) and for testbench variables. Never assign the same variable with both kinds.

Q57. Compare the four unpacked array types and say when you would use each.

Property	Fixed-size	Dynamic	Associative	Queue
Declaration	`int a [8];`	`int d [];`	`int aa [string];`	`int q [$];`
Size	Fixed at compile time	Set with `new[]` at run time	Grows as entries are written	Grows and shrinks with push/pop
Index	0..N-1, contiguous	0..N-1, contiguous	Any type, sparse	0..N-1, contiguous
Memory	All elements allocated	All elements allocated	Only written entries	Current elements
Insert/delete in the middle	No	Only by reallocating	Yes, by key	Yes
Typical use	Known-size data, RTL memories	Payload whose length is random per packet	Sparse memory model, lookup by ID or name	Expected-transaction list in a scoreboard, FIFO model

Q58. What does `b = a;` do when `a` and `b` are dynamic arrays? It copies all the elements. `b` is resized to the size of `a` and receives its own copy of the data, so a later change to `a[0]` does not affect `b[0]`. The same is true for queues and associative arrays. Arrays are not handles.

Q59. What does `b = a;` do when `a` and `b` are arrays of class handles? The array is copied, but each element is a handle, so only the handles are copied. `a[0]` and `b[0]` refer to the same object. To get independent objects, each element must be copied: `foreach (a[i]) b[i] = a[i].copy();`.

Q60. How do you delete elements from a queue while looping over it? Deleting inside a forward `foreach` shifts the following elements down, so the element after each deleted one is skipped. Loop backwards, or build the result with a locator method:

```
for (int i = q.size() - 1; i >= 0; i--)
  if (q[i] % 2 == 0) q.delete(i);          // safe: indices below i are unaffected

q = q.find with (item % 2 != 0);            // alternative: keep only the odd values
```

Q61. How do the array locator methods use the `with` clause? The `with` clause is an expression evaluated for each element; `item` is the element and `item.index` is its index. The method returns the elements (or indices) for which the expression is true.

```
int q [$] = '{3, 8, 1, 8, 5};
int r [$];
r = q.find with (item > 4);                 // {8, 8, 5}
r = q.find_first_index with (item == 8);    // {1}
r = q.find_index with (item.index % 2 == 0);// {0, 2, 4}
r = q.find_last with (item < 5);            // {1}
```

`find`, `find_index`, `find_first` and the other `find_` methods require a `with` clause. `min`, `max`, `unique`, `sum`, `sort` accept one optionally.

Q62. An array is declared `bit [7:0] b [4]` and holds 200, 100, 50, 10. What does `b.sum()` return, and how do you get 360? 104. `sum()` returns a value of the element type, here 8 bits, so 360 wraps to 360 - 256 = 104. Widen each element in a `with` clause: `b.sum() with (int'(item))` returns 360. The same trap applies to `product()` and to counting matches: `b.sum() with (int'(item > 60))` counts the elements greater than 60 (without the cast the count is 1 bit wide).

Q63. How do you sort a queue of transaction objects by one field? With the `with` clause of `sort` or `rsort`: `txn_q.sort() with (item.id);` sorts in ascending order of `id`; `rsort` gives descending order. Several keys can be combined with a concatenation: `sort() with ({item.op, item.a})`.

Q64. How do you iterate over an associative array? With `foreach`, or with `first()` and `next()`, which take the index variable by reference and return 0 when there is no (further) entry:

```
int    score [string];
string name;
if (score.first(name))
  do $display("%s = %0d", name, score[name]);
  while (score.next(name));
```

Entries are visited in ascending index order (numeric order for integral indices, lexical order for strings). `last()` and `prev()` iterate backwards.

Tasks and Functions

Q65. What is the difference between `ref` and `const ref`, and why does `ref` need an automatic routine? A `ref` argument is a reference to the caller's variable: reads and writes act on the original. `const ref` passes the reference but forbids writing, which gives the speed of no copying with the safety of an input. A `ref` argument is allowed only in a routine with automatic lifetime (a class method, or a task or function declared `automatic`), because a static routine has one permanent copy of its arguments and could keep a reference to a variable that no longer exists.

Q66. Two threads call the same task at the same time and the results are corrupted. What is the likely cause? The task is static (the default in a module, interface or package), so both calls share one copy of the

arguments and local variables. The second call overwrites the arguments of the first while it is waiting at a delay. Declaring the task `automatic` gives each call its own storage.

```
task automatic send(input int id, input int delay);
  #delay $display("[%0t] id=%0d", $time, id);
endtask
```

Q67. Can a function call a task? Can a function consume time? No to both. A function must return in zero time, so it may not contain `#`, `@`, `wait`, or a call to a task (which might contain them). The one exception: a function may start a time-consuming thread with `fork`...`join_none`, because the function itself still returns immediately.

Q68. In what ways can a function return a value? (1) `return expr;`. (2) By assigning to the function name: `my_func = expr;` (the Verilog style). (3) Through `output`, `inout` or `ref` arguments, which is how a function returns more than one value. If no value is assigned, the function returns the default of its return type.

Classes

Q69. What is the difference between a shallow copy and a deep copy? A shallow copy, `b = new a;`, creates a new object and copies every property of `a` into it. Properties that are handles are copied as handles, so the nested objects are shared between `a` and `b`. A deep copy also creates new copies of the nested objects, so nothing is shared. A deep copy must be written by hand in a `copy()` method:

```
function alu_pkt copy();
  alu_pkt c = new();
  c.len = this.len;            // plain property: copy the value
  c.hdr = this.hdr.copy();     // handle property: copy the object it refers to
  return c;
endfunction
```

Q70. What is the difference between `b = a;` and `b = new a;` for class handles? When is an object destroyed? `b = a;` copies the handle: there is still one object, now with two handles. `b = new a;` creates a second object (shallow copy). SystemVerilog has no `delete` for objects; an object's memory is reclaimed automatically when no handle refers to it any more, for example after `a = null; b = null;`.

Q71. In what order do constructors run in an inheritance chain, and what is the rule for `super.new()`? The base-class constructor runs first, then the property initialisers of the derived class, then the body of the derived constructor. If the derived constructor calls `super.new(...)` explicitly, that call must be the first statement. If it does not, the compiler inserts `super.new()` with no arguments, which is an error if the base constructor has arguments without defaults.

Q72. What are the rules for virtual methods? (1) Once a method is declared `virtual` in a class, it is virtual in all derived classes, whether or not they repeat the keyword. (2) An overriding method must have the same return type, argument types and argument count as the base method. (3) The call is resolved from the type of the object when made through a handle. (4) Constructors and static methods cannot be virtual. (5) Properties are never virtual.

Q73. What is downcasting and why does it need `$cast`? Assigning a base-class handle to a derived-class handle. It is legal only if the object really is of the derived type (or a type derived from it), which is known only at run time, so a plain assignment is a compile error and `$cast` performs the check.

```
base  b;
child c1 = new(), c2;
b = c1;                          // upcast: always legal, no cast needed
if (!$cast(c2, b)) $error("b does not refer to a child object");
```

Q74. What is the difference between overriding and overloading? Does SystemVerilog support both? Overriding is redefining a base-class method in a derived class with the same name and signature. Overloading is having several methods with the same name but different argument lists in one scope. SystemVerilog supports overriding but not overloading: two methods with the same name in one class are an error. Default argument values give a similar convenience.

Q75. What are the restrictions on a static method? It can be called without an object (`alu_txn::get_count()`), so it has no `this` . It can access only static properties and call only static methods of its class, and it cannot be virtual. Note the difference from `static task` in a module-level sense: `static function` in a class makes the method belong to the class, while `function static` would make its variables static.

Q76. What is an `extern` method? A method whose prototype is declared inside the class with `extern` and whose body is written outside the class using the scope operator: `function void alu_txn::print(); ... endfunction` . It keeps the class declaration short and readable.

Q77. What is a parameterised class? A class with parameters for values or types, so one definition serves many types: `class stack #(type T = int, int DEPTH = 8);` . Each distinct set of parameter values is a separate specialisation, a different type: `stack #(alu_txn) s;` . `mailbox #(T)` is a built-in example.

Randomization and Constraints

Q78. What is the difference between the implication operator and `if-else` in a constraint? Are they one-directional? `a -> b` means "if `a` is true then `b` must be true". `if (a) b; else c;` is the same as `(a -> b) && (!a -> c)` ; the `else` gives a constraint for the false case as well. Neither is one-directional: constraints are relations solved together, so `a -> b` also means that if `b` is false then `a` must be false. With `mode == 1 -> len < 10` , a solution with `len == 20` forces `mode != 1` .

Q79. What does `solve ... before` do? It changes the order in which the solver picks values, and therefore the probability of each solution. It does not change which solutions are legal. In `constraint c { flag -> data == 0; solve flag before data; }` with a 1-bit `flag` and an 8-bit `data` , without the ordering `flag == 1` appears in only 1 of 257 solutions; with it, `flag` is chosen first and is 1 half of the time.

Q80. How does a soft constraint get overridden? Any hard constraint that contradicts it wins, and the soft constraint is dropped without a failure. Among soft constraints, the one with higher priority wins: constraints in a `randomize() with {}` block have priority over those in the class, a derived class over its base class, and a later declaration over an earlier one. `disable soft x;` discards all lower-priority soft constraints on `x` .

Q81. What does the `unique` constraint do? It forces a group of variables or all elements of an array to have different values: `constraint u_c { unique {arr}; }` or `unique {a, b, c};` . It replaces the nested `foreach` loops that were needed before IEEE 1800-2012.

Q82. What is the difference between restricting a variable with `inside` and with `dist` ? `inside` only defines the legal set; all values in it are equally likely (subject to other constraints). `dist` defines the legal set and a weight for each value or range, so some values are generated more often. A value with weight 0 or not listed in the `dist` is never generated. `dist` cannot be applied to a `randc` variable.

Q83. What is `randomize() with` and what is `local::` for? `randomize() with { ... }` adds constraints for one call only, on top of the class constraints: `t.randomize() with { a < 10; op == ALU_ADD; }`. Names inside the braces are looked up in the object first. If a variable in the calling scope has the same name as a property, `local::` selects the one from the calling scope: `t.randomize() with { a == local::a; }`.

Q84. How do you constrain the size and the elements of a dynamic array? Constrain `size()` and use `foreach` for the elements. The solver chooses the size first and then solves the elements.

```
rand bit [7:0] payload [];
constraint payload_c {
  payload.size() inside {[1:16]};
  foreach (payload[i]) payload[i] != 8'h00;
}
```

Always constrain the size; an unconstrained size can be huge or zero.

Q85. How do you randomize a variable that is not a class property? With the scope randomize function, `std::randomize(x) with { x inside {[1:9]}; };`, which works on any variables visible at the call and returns 1 or 0 like the class method. For simple cases use `$urandom()` (32-bit unsigned) or `$urandom_range(max, min)`. Avoid `$random`: it is signed and its sequence is not tied to the thread's random stability.

Q86. How do constraints behave under inheritance? A derived class inherits all constraint blocks of its base class. A constraint block in the derived class with a new name is added to them. A block with the **same name** as a base-class block replaces (overrides) it. This is how a test narrows or changes stimulus: extend the transaction, override or add a constraint, and make the generator create the extended class.

Inter-Process Communication

Q87. Compare mailbox, semaphore and event.

	Mailbox	Semaphore	Event
Purpose	Pass data between processes	Control access to a shared resource	Signal that something happened
Carries data	Yes	No (only a key count)	No
Blocking calls	`put` (if full), `get`, `peek`	`get`	`@e`, `wait(e.triggered)`
Remembers what happened	Yes, items are stored	Yes, keys are stored	No (only `triggered`, for the current time step)
Typical use	Generator → driver, monitor → scoreboard	Two generators sharing one driver	"Generator done", "reset finished"

Q88. What is the difference between `@e` and `wait (e.triggered)`? `@e` waits for the next trigger after the statement starts executing. If the trigger happens in the same time step but before the `@e` is reached, it is missed and the process may wait forever. `e.triggered` is true from the trigger until the end of that time step, so `wait (e.triggered)` passes whether the trigger came just before or comes later. Use `wait (e.triggered)` when the trigger and the wait can occur in the same time step. Do not put `wait (e.triggered)` in a zero-delay loop: it stays true for the whole time step and the loop never advances time.

Q89. What is the difference between `disable fork`, `disable` with a block name, and `wait fork`? `disable fork;` kills all active threads that were spawned by the calling process (and their descendants). `disable`

`block_name;` kills the named block wherever it is running, including in other instances of the same task. `wait fork;` kills nothing: it blocks the calling process until all the threads it spawned have finished. A typical pattern is `fork ... join_any` followed by `disable fork;` to implement a timeout.

Q90. What is the difference between `->` and `->>`? `-> e` triggers the event immediately; processes already waiting on it become ready at once. `->> e` is the nonblocking trigger: the event is triggered in the NBA region of the current time step. It is used to avoid a race when the trigger and the `@e` are executed in the same time step in unknown order, because by the NBA region every process has reached its `@e`.

Q91. What is the difference between a bounded and an unbounded mailbox, and between a parameterised and a generic one? `new()` creates an unbounded mailbox: `put()` never blocks. `new(n)` creates a bounded one holding at most `n` items: `put()` blocks when it is full, which throttles the producer. `mailbox #(alu_txn) m;` is parameterised: it accepts only that type and type errors are found at compile time. A plain `mailbox m;` is generic and accepts any type; a `get()` into a variable of the wrong type is a run-time error.

Q92. What is wrong with starting threads in a loop like this, and how is it fixed?

```
for (int i = 0; i < 3; i++)
  fork
    $display("thread %0d", i);
  join_none
```

Threads started by `join_none` do not run until the parent blocks, which is after the loop has finished, and they all share the single variable `i`, which is then 3. All three print 3. Declare an automatic copy inside the `fork`, which is initialised when the thread is created:

```
for (int i = 0; i < 3; i++)
  fork
    automatic int k = i;
    $display("thread %0d", k);
  join_none
```

Q93. What happens if a process asks a semaphore for more keys than exist, or puts back keys it never took? `get(n)` blocks until `n` keys are available; if that never happens the process waits forever. `put(n)` simply adds `n` keys, even if this makes the count larger than the number the semaphore was created with. The semaphore does not track ownership, so a missing `put()` (deadlock) or an extra `put()` (two users at once) are bugs the language will not catch.

Interfaces and the Testbench

Q94. Why can a class not use an interface directly, and how does a virtual interface solve it? An interface instance is a static design element created at elaboration, like a module instance. A class object is created at run time. A class therefore cannot instantiate an interface or have ports. A virtual interface is a variable that holds a reference to an interface instance; the class declares it, the testbench top (static world) passes the instance in, and the class reads and drives signals through it.

Q95. How does the virtual interface get from `tb_top` to the driver? `tb_top` instantiates `alu_if vif (clk);` and connects it to the DUT. It then passes the instance to the test, usually as a constructor argument: `test = new(vif);`. The test passes it to the environment, and the environment passes it to the driver and monitor, each storing it in its own `virtual alu_if vif;`. In UVM the same job is done by `uvm_config_db#(virtual alu_if)::set` in the top and `get` in the component.

Q96. What are clocking block skews and what are the defaults? The input skew says how long before the clock event an input is sampled; the output skew says how long after the clock event an output is driven. The defaults are `input #1step` (sample the value the signal had just before the clock edge, in the Preponed region) and `output #0` (drive in the Re-NBA region of the clock's time step, after the DUT has sampled). They can be changed: `default input #2ns output #1ns;`.

Q97. What is a program block and is it still used? A `program` is a testbench container whose code executes in the Reactive region, after the design has settled in the Active and NBA regions, which separates testbench and design activity. Simulation ends implicitly when all program blocks finish. Class-based testbenches and UVM normally do not use it; a module top with clocking blocks gives the same protection, and UVM guidance is to avoid program blocks.

Q98. What is the difference between a driver and a monitor? The driver is active: it receives transactions, drives the DUT input signals and follows the protocol timing. The monitor is passive: it never drives anything; it samples the signals (inputs and outputs), rebuilds transactions and sends them to the scoreboard and coverage. The scoreboard must take its data from the monitor, not from the generator or driver, so that what is checked is what really appeared on the pins.

Coverage and Assertions

Q99. What is the difference between `ignore_bins` and `illegal_bins`? Both remove values from coverage. Values in `ignore_bins` are simply not counted and not required. Values in `illegal_bins` are also excluded, and in addition the simulator reports a run-time error if one of them is sampled. Use `ignore_bins` for cases that are not interesting, `illegal_bins` for cases that must never occur (though an assertion is the better place for such a check).

Q100. How many automatic bins does a coverpoint get? If a coverpoint has no bins, one bin is created per value, up to `option.auto_bin_max`, whose default is 64. A coverpoint on a 4-bit variable gets 16 bins; a coverpoint on a 16-bit variable gets 64 bins, each covering 1024 values. A coverpoint on an enum gets one bin per named value.

Q101. What is cross coverage and what is `binsof`? A cross counts the combinations of the bins of several coverpoints: `cross cp_op, cp_a_range;` with 8 and 3 bins has 24 cross bins. `binsof` selects bins of a coverpoint inside the cross in order to group or exclude combinations:

```
cx: cross cp_op, cp_a {
  ignore_bins no_not = binsof(cp_op) intersect {ALU_NOT};
}
```

Q102. Name some coverage options. `option.per_instance` (keep coverage separately per covergroup instance), `option.at_least` (hits needed to count a bin as covered, default 1), `option.auto_bin_max` (default 64), `option.weight`, `option.goal` (default 100), `option.comment`, `option.cross_auto_bin_max`. `type_option` holds the options that apply to the covergroup type as a whole, such as `type_option.merge_instances`.

Q103. What are transition bins? Bins that count sequences of values on successive samples of a coverpoint: `bins start = (IDLE => FETCH);`, `bins loop3 = (EXEC [* 3]);` (three consecutive samples), `bins any = (IDLE => FETCH, EXEC);`. They are used to cover state-machine transitions and command sequences.

Q104. What do `$rose`, `$fell`, `$stable`, `$changed` and `$past` do? They are sampled-value functions that compare the value of an expression at the current clock tick with its value at the previous tick. `$rose(x)` is true if the least significant bit of `x` changed to 1; `$fell(x)` if it changed to 0; `$stable(x)` if the whole value did not

change; `$changed(x)` if it did. `$past(x, n)` returns the value `x` had `n` ticks earlier (default 1). All of them use values sampled just before the clock edge.

Q105. What does `disable iff` do? It gives a condition, normally reset, under which the property is not checked: `assert property (@(posedge clk) disable iff (!rst_n) req |=> ack);`. While the condition is true, attempts in progress are abandoned and no failure is reported. The condition is asynchronous: it is not sampled on the clock.

Q106. What is the difference between `assert`, `assume` and `cover`? `assert property` checks that the property holds and reports a failure if it does not. `assume property` states a condition on the environment: in simulation it is checked like an assertion, in formal verification it constrains the inputs. `cover property` does not check anything; it records that the described behaviour occurred at least once.

Q107. What do the sequence operators `##n`, `[*n]`, `[->n]` and `[=n]` mean? `a ##2 b`: `b` is true two clock ticks after `a`. `a ##[1:3] b`: one to three ticks later. `a [*3]`: `a` true on three consecutive ticks. `a [->3]` (goto repetition): `a` true three times, not necessarily consecutively, the match ending at the third occurrence. `a [=3]` (non-consecutive repetition): `a` true three times, and the match may extend past the third occurrence while `a` stays false.

Packages, DPI and UVM

Q108. What is the difference between importing a package and using `` `include ``? `` `include `` pastes the text of a file at that point; the included declarations belong to whatever scope contains the directive, and including the same class file in two packages creates two different, incompatible types. A package is compiled once and defines one set of types that every importer shares. The usual practice is to `` `include `` the class files inside one package and `import` that package everywhere.

Q109. What is DPI? The Direct Programming Interface: a way to call C functions from SystemVerilog and SystemVerilog functions and tasks from C without the complexity of PLI/VPI. `import "DPI-C" function int c_model(input int a);` makes a C function callable; `export "DPI-C" function sv_func;` makes a SystemVerilog function callable from C. It is used for reference models written in C or C++.

Q110. How do the main UVM mechanisms map to plain SystemVerilog?

UVM	What it is built from
`uvm_component`, `uvm_object`	Base classes; your classes extend them (inheritance)
Factory (`type_id::create`, overrides)	Polymorphism: a base handle receives an object of a derived class chosen at run time
Phases (`build_phase`, `run_phase` ...)	Virtual methods called in a fixed order by the library; `run_phase` tasks are forked in parallel
`uvm_config_db`	An associative-array database, used mainly to pass virtual interfaces and settings
Sequence → sequencer → driver	Generator → mailbox → driver, with a handshake (`get_next_item`, `item_done`)
TLM ports and analysis ports	Method calls between components, replacing direct mailbox handles
Objections	A counter that decides when the test ends, replacing the `gen_done` event
`uvm_agent`	Driver, monitor and sequencer for one interface, grouped for reuse

Advanced Questions

These questions are asked to candidates who answered the basics well. They probe the simulation scheduler, the constraint solver, the details of the object model, and the sampling rules of assertions and coverage.

Scheduling and Timing

Q111. What are the event regions of a SystemVerilog time step? Within one time step the simulator processes events in a fixed order of regions:

```
from previous time step
          |
     Preponed      sample values for concurrent assertions and #1step inputs
          |
+--->  Active      blocking assignments, continuous assignments, RHS of
|         |        nonblocking assignments, $display, module code
|     Inactive     statements delayed with #0
|         |
|       NBA        nonblocking assignment updates
|         |        (new events go back to Active)
|    Observed      concurrent assertions are evaluated
|         |
|    Reactive      program block code, assertion action blocks
|         |
|   Re-Inactive    #0 in programs
|         |
+---  Re-NBA       nonblocking assignments from programs, clocking block drives
          |
     Postponed     $strobe, $monitor
          |
  to next time step
```

The loop Active → Inactive → NBA repeats until no events remain, then the Observed and Reactive regions run; if they create new design events the simulator goes back to Active. The purpose is to separate design activity (Active, NBA) from testbench and assertion activity (Observed, Reactive) so that the testbench sees settled values.

Q112. Why do nonblocking assignments prevent races in sequential RTL? Two flip-flops clocked by the same edge are two `always_ff` blocks that the simulator may execute in either order. With blocking assignments, the second block would read the value already updated by the first. With nonblocking assignments, every right-hand side is evaluated in the Active region using the old values, and all left-hand sides are updated afterwards in the NBA region, so the order of execution of the blocks does not matter. This matches real hardware, where all flip-flops sample at the edge and change after it.

Q113. Why is there a race between a testbench and the DUT, and how does a clocking block remove it? If the testbench does `@(posedge clk); a = value;` then, in the Active region of the clock edge, both the DUT's `always_ff` (which samples `a`) and the testbench assignment are scheduled, in an order the language does not define. The DUT may capture the old or the new value. Sampling a DUT output at the edge has the same problem. A clocking block fixes both directions: inputs are sampled with skew `#1step`, that is in the Preponed region, before anything changes at the edge; outputs are driven in the Re-NBA region, after the design has sampled. The testbench therefore behaves like a flip-flop connected to the DUT.

Q114. What does `#0` do, and why is it discouraged? `#0` suspends the process and reschedules it in the Inactive region of the same time step, so it resumes after all currently active events. It is sometimes used to "let

the other process go first". It is discouraged because it only moves the race: if two processes both use `#0`, the order is undefined again, and code that depends on it breaks when unrelated code changes. Use nonblocking assignments, clocking blocks, `wait (e.triggered)` or `->>` instead.

Q115. What is `#1step`? A special delay meaning "one unit of the global time precision, just before the current time step". As an input skew it means the value is taken from the Preponed region of the time step containing the clock event, which is the value the signal had at the end of the previous time step. It is the default input skew of a clocking block and is also how concurrent assertions sample their signals.

Q116. In which regions are concurrent assertions sampled, evaluated and reported? The signals are sampled in the Preponed region. The property is evaluated in the Observed region, after the design has settled. The pass and fail action blocks execute in the Reactive region. A consequence: an assertion clocked on `posedge clk` sees the values from just before the edge, not the values assigned by the flip-flops at that edge.

Q117. What do these print, and why?

```
initial begin
  x = 1;
  x <= 2;
  $display("display: %0d", x);
  $strobe ("strobe : %0d", x);
end
```

`display: 1` then `strobe : 2`. `$display` runs in the Active region, before the nonblocking update has happened. `$strobe` prints in the Postponed region, after the NBA region has set `x` to 2. This is why `$strobe` or `$monitor` is used to print the final values of a time step.

Q118. What is the difference between `@(vif.drv_cb)` and `@(posedge vif.clk)` in a driver? `@(posedge vif.clk)` resumes the driver in the Active region of the clock edge, where it can race with the DUT. `@(vif.drv_cb)` waits for the clocking event of the clocking block; the process resumes in the Observed region, after the clocking block has sampled its inputs and after the design has run, so values read through `vif.drv_cb.signal` are the newly sampled ones and there is no race. When a clocking block is used, all waiting should be on the clocking block event, not on the raw clock; mixing the two brings the race back.

The Constraint Solver

Q119. How does the solver choose values when several variables are constrained together? All active constraints and all active random variables are solved together as one problem. The solver finds the set of all legal combinations (the solution space) and picks one combination with uniform probability, unless `dist` or `solve ... before` changes the distribution. Example: `rand bit [1:0] a, b; constraint c { a < b; }` has six solutions, (0,1) (0,2) (0,3) (1,2) (1,3) (2,3), each with probability 1/6. So `a == 0` has probability 1/2 and `a == 2` has probability 1/6: the variables are not individually uniform.

Q120. Can `solve ... before` make `randomize()` fail or succeed? No. It only changes the order of selection and therefore the probabilities. The set of legal solutions is exactly the same with or without it. If the constraints are contradictory, no ordering helps. It also cannot be used with `randc` variables, which are always solved first.

Q121. What happens when a function is called inside a constraint? The function's arguments are solved first, without regard to the constraints that use the function's result; the function is then evaluated and its return value is treated as a constant in the remaining constraints. Two consequences: the relation is no longer bidirectional (the result cannot influence the arguments), and randomization can fail even though a solution exists. Such functions must be automatic, must not have side effects and must not have `output` or `ref` arguments.

Q122. How does `randc` work internally and what are its limits? The solver computes a random permutation of the legal values of the variable and returns the next entry on each call; when the permutation is exhausted a new one is made. `randc` variables are solved before `rand` variables. Limits: the type may be at most a tool-defined size (at least 16 bits is guaranteed), `dist` cannot be applied to it, and if the constraints change between calls so that the set of legal values changes, the cycle restarts. Constraints that tie a `randc` variable to other random variables can make the cyclic behaviour impossible to keep.

Q123. Several soft constraints conflict. Which one wins? The one with the highest priority. Priority increases in this order: constraints in a base class, constraints in a derived class, constraints declared later in the same class, and finally inline constraints in `randomize() with {}` (highest). The solver adds soft constraints from highest to lowest priority and discards any that would make the problem unsolvable. A hard constraint always beats any soft constraint.

Q124. What do `randomize(null)` and `randomize(a, b)` do? `obj.randomize(null)` assigns nothing: every variable is treated as a state variable and the call only checks whether the current values satisfy the constraints, returning 1 or 0. It is used as a checker. `obj.randomize(a, b)` randomizes only the listed properties for this call (even properties not declared `rand`), treating all others as state variables.

Q125. In what order are the size and the elements of a random dynamic array solved? The size is solved first, the array is resized, and then the `foreach` constraints are applied to the elements. A constraint that tries to make the size depend on the element values, such as "the size equals the number of non-zero elements", cannot work as written.

Q126. What exactly happens when `randomize()` fails? It returns 0, the random variables keep the values they had before the call, `post_randomize()` is not called (`pre_randomize()` has already run), and most simulators print a warning naming the conflicting constraints. Simulation continues. If the return value is not checked, the test carries on with stale stimulus, which is why every call should be wrapped in a check.

Q127. What is random stability, and how do you reproduce a failing random test? Each thread and each object has its own random number generator, seeded from its parent when it is created. Random stability means that the values drawn by one thread or object do not depend on what other threads do. A failing test is reproduced by rerunning with the same simulator seed (for example `+ntb_random_seed=` or `-sv_seed`, depending on the tool) and the same code. Adding or reordering object creations or forks before the point of interest changes the seeds handed out afterwards, so a "small" testbench edit can change the stimulus. An object can be reseeded explicitly with `obj.srandom(seed)`.

Classes and Memory

Q128. How is object memory managed? Can there be a memory leak? Objects are allocated by `new` and reclaimed automatically when no handle refers to them. There is no explicit free. Leaks in the usual sense happen when handles are kept unintentionally: a scoreboard queue or associative array that is never emptied keeps every transaction of the test alive, and memory grows with simulation length.

Q129. Are class properties polymorphic like virtual methods? No. Property access is resolved from the type of the handle at compile time. If a base class and a derived class both declare `int x`, then through a base handle `b.x` is the base property even when the object is of the derived class. Only virtual methods are resolved from the object. Redeclaring a base property in a derived class (hiding) is legal but almost always a mistake.

Q130. Why is `copy()` normally a virtual method that returns the base type? So that a component holding a base-class handle can copy an object without knowing its real type. Each derived class overrides `copy()` to

create an object of its own type and copy its extra fields; a call through the base handle then produces a complete copy of the derived object. If `copy()` were not virtual, copying through a base handle would create a base object and lose the derived fields. Because the signature of an override must match, the return type stays the base type and the caller uses `$cast` if it needs the derived handle.

Q131. Does SystemVerilog support multiple inheritance? What about `super.super`? A class can extend only one class, and `super.super.method()` is not allowed; a class can reach only its direct base. IEEE 1800-2012 added `interface class`: a class containing only pure virtual method prototypes (and types and parameters). A class can `implements` several interface classes, which gives multiple inheritance of the method contract but not of code or data.

Q132. How do static members behave in a parameterised class? Each specialisation is a separate class with its own static members. In `class counter #(type T = int); static int n; endclass`, `counter#(int)::n` and `counter#(bit)::n` are different variables. To share one variable among all specialisations, put it in a non-parameterised base class.

Q133. Does a shallow copy (`b = new a;`) call the constructor? No. Memory is allocated and the property values are copied from `a`, but the `new()` function of the class is not executed. Anything the constructor does, such as incrementing a static object counter or creating nested objects, does not happen for the copy.

Process Control

Q134. What is the `process` class? A built-in class that gives a handle to a thread. `process::self()` returns the handle of the calling thread. Methods: `status()` returns `FINISHED`, `RUNNING`, `WAITING`, `SUSPENDED` or `KILLED`; `kill()` terminates the thread and its children; `await()` blocks until the thread finishes; `suspend()` and `resume()` pause and continue it. It gives finer control than `disable fork`, which cannot select a single thread.

```
process p;
fork
  begin p = process::self(); do_work(); end
join_none
wait (p != null);
#100 if (p.status() != process::FINISHED) p.kill();
```

Q135. `disable fork` killed a thread it was not supposed to. Why, and how is that prevented? `disable fork` kills every thread spawned by the calling process, including threads started much earlier with `join_none` (for example a background monitor), not only those of the nearest `fork`. Wrap the timeout logic in its own `fork`...`join` so that it runs in a new process whose only children are the ones to be killed:

```
fork
  begin                       // isolating process
    fork
      wait_for_response();
      #1000;
    join_any
    disable fork;             // kills only the two threads above
  end
join
```

Q136. A thread started with `join_none` uses a local variable of the task that started it, and the task returns. Is that safe? Yes, for automatic variables. An automatic variable lives as long as any thread that can reference it, so the storage of the enclosing scope is kept until the spawned threads finish. The risk is different: all threads see the same variable, and its value when they run may not be the value it had when they were created (the loop-

variable trap of Q92). A `ref` argument, however, may not be used inside `fork` ... `join_none` or `join_any`, because the referenced variable could cease to exist.

Q137. What happens when one event variable is assigned to another? Event variables are handles to synchronisation objects. After `e2 = e1;` both names refer to the same event: triggering either wakes processes waiting on either. Processes that were already blocked on the old `e2` object are not woken by later triggers, because nothing refers to that object any more. Assigning `null` to an event variable disconnects it. This handle behaviour is what allows an event to be shared between classes by passing it to a constructor.

Q138. Several processes are blocked on `get()` of the same mailbox or semaphore. Who is served first? A mailbox serves waiting processes in the order in which they started waiting (FIFO). For a semaphore the standard also specifies a FIFO waiting queue, but a request for few keys may be satisfied before an earlier request for more keys that cannot yet be met, and simulators differ in the detail. Do not write testbenches that depend on fairness between waiters; if order matters, enforce it explicitly.

Coverage Semantics

Q139. What are the rules for a covergroup declared inside a class? An embedded covergroup is declared in the class and must be constructed in the class constructor: `cg = new();` (only one instance per object, and the name of the covergroup is used as the variable). It can refer to the class properties directly. It is sampled either by its own event, `covergroup cg @(posedge clk);`, or, more commonly in a class, by calling `cg.sample()` when a transaction is complete. If the constructor does not `new` it, `sample()` is a null-handle error.

Q140. What is the difference between `option` and `type_option`, and between `get_coverage()` and `get_inst_coverage()`? `option` settings apply to one instance of a covergroup (or one coverpoint); `type_option` settings apply to the covergroup type across all instances. `cg.get_inst_coverage()` returns the coverage percentage of that instance alone (meaningful when `option.per_instance = 1`); `cg.get_coverage()` returns the coverage of the type, combined over all instances. `$get_coverage()` returns the overall figure for all covergroups in the simulation.

Q141. How are X and Z handled in coverage, and what are wildcard bins? Automatically created bins cover only two-state values; a sampled value containing X or Z does not fall into any automatic bin. In an explicit bin, a value is counted only on an exact match, unless the bin is declared `wildcard`, in which case X, Z and `?` in the bin definition are don't-care positions: `wildcard bins odd = {4'b???1};` counts every odd value.

Assertion Semantics

Q142. What is a vacuous success? When the antecedent of an implication does not match, the property is true by definition, although nothing was actually checked. `req |-> ack` passes vacuously on every clock where `req` is 0. Tools do not count vacuous successes in assertion coverage. The danger is an assertion whose antecedent can never be true: it never fails and gives false confidence, which is why each assertion should have a matching `cover property` for its antecedent.

Q143. How many evaluation attempts of a concurrent assertion can be active at once? A new attempt starts at every clock tick, independently of earlier attempts that are still in progress. For `req |-> ##[1:5] ack`, if `req` is high on three consecutive clocks there are three overlapping attempts, and a single `ack` may complete all of them. This matters when the intent is "one `ack` per `req`": the simple property does not check that, and a counter or a local variable is needed.

Q144. What does `$past(x)` return on the first clock tick, and what is its gating argument? On ticks before enough history exists, `$past` returns the default initial value of the expression's type (X for four-state types, 0 for two-state). Properties using `$past` therefore usually need `disable iff` during reset, or an explicit guard. The full form is `$past(expr, n, gate, clock)`: with a gating expression, only clock ticks on which `gate` was true are counted as "past" ticks.

Q145. What are `throughout`, `within`, `intersect`, `first_match` and sequence local variables? `cond throughout seq`: `cond` must hold on every tick of `seq`. `s1 within s2`: `s1` must occur entirely inside the span of `s2`. `s1 intersect s2`: both match, starting and ending at the same ticks. `first_match(seq)`: of all possible matches of a sequence with ranges, only the earliest is used. A local variable is declared inside a sequence or property and captures a value at one point for comparison later:

```
property data_through_p;
  logic [7:0] d;
  @(posedge clk) (wr_en, d = din) |-> ##[1:$] (rd_en && dout == d);
endproperty
```

Q146. What is `bind`? A statement that instantiates a module or interface (typically one containing assertions or coverage) inside another module without editing that module's source: `bind sync_fifo fifo_sva u_sva (.*);`. The bound instance behaves as if it had been written inside the target, with access to its signals. Verification engineers use it to attach assertions to RTL they do not own, including VHDL blocks.

Output-Prediction Questions

For each program, write down the exact output before reading the answer. Go statement by statement and keep track of the state. Every program here has one defined result: parallel threads use different delays, and no unseeded random values are printed.

Arrays and Strings

Q147. Queue operations.

```
module q147;
  int q [$] = '{1, 2, 3};
  initial begin
    q.push_back(4);
    q.push_front(0);
    q.insert(2, 9);
    void'(q.pop_back());
    q.delete(1);
    $display("%0d %0d %0d %0d size=%0d", q[0], q[1], q[2], q[3], q.size());
    $display("%0d %0d", q[$], q[$-1]);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
0 9 2 3 size=4
3 2
```

The queue goes through these states: {1,2,3} → push_back(4) {1,2,3,4} → push_front(0) {0,1,2,3,4} → insert(2,9) {0,1,9,2,3,4} → pop_back {0,1,9,2,3} → delete(1) {0,9,2,3}. `q[$]` is the last element and `q[$-1]` the one before it.

Q148. Dynamic array allocation.

```
module q148;
  int d [];
  initial begin
    d = new[3];
    foreach (d[i]) d[i] = i + 1;
    d = new[5](d);
    $display("%0d %0d %0d", d.size(), d[2], d[4]);
    d = new[2];
    $display("%0d %0d", d.size(), d[0]);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
5 3 0
2 0
```

`new[5](d)` allocates five elements and copies the old three: {1,2,3,0,0}. `new[2]` without the copy argument allocates a fresh array filled with the default value 0; the old contents are discarded.

Q149. Associative array iteration.

```
module q149;
  int    aa [string];
  string k;
  initial begin
    aa["b"] = 2;
    aa["a"] = 1;
    aa["c"] = 3;
    aa.delete("b");
    if (aa.first(k))
      do $display("%s=%0d", k, aa[k]);
      while (aa.next(k));
    $display("%0d %0d", aa.num(), aa.exists("b"));
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
a=1
c=3
2 0
```

Entries are visited in the sorted order of the index, not the order of insertion. After the delete only two entries remain and `exists("b")` is 0.

Q150. Array locator and ordering methods.

```
module q150;
  int a [5] = '{4, 1, 3, 1, 5};
  int r [$];
  initial begin
    r = a.find with (item > 2);
    $display("%0d: %0d %0d %0d", r.size(), r[0], r[1], r[2]);
    r = a.find_index with (item == 1);
    $display("%0d %0d", r[0], r[1]);
    r = a.unique();
    $display("%0d", r.size());
    r = a.max();
    $display("%0d %0d", r[0], a.sum());
    a.rsort();
    $display("%0d %0d", a[0], a[4]);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
3: 4 3 5
1 3
4
5 14
5 1
```

`find` returns the matching elements in index order. `find_index` returns the indices of the two 1s. `unique` returns one copy of each distinct value: 4, 1, 3, 5. `max` returns a queue with one element. The sum is 14. `rsort` sorts in descending order: {5,4,3,1,1}.

Q151. The width of `sum()`.

```
module q151;
  bit [7:0] b [4] = '{200, 100, 50, 10};
  initial begin
    $display("%0d", b.sum());
    $display("%0d", b.sum() with (int'(item)));
    $display("%0d", b.sum() with (int'(item > 60)));
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
104
360
2
```

`sum()` has the type of the elements, 8 bits, so 360 wraps to 104. Casting each element to `int` makes the sum 32 bits wide. The last line counts the elements greater than 60 (200 and 100).

Q152. String methods.

```
module q152;
  string s = "Hello";
  string n = "42abc";
  initial begin
    s = {s, " SV"};
    $display("%0d|%s|%s|%0d", s.len(), s.toupper(), s.substr(0, 3), s.compare("Hello SV"));
    $display("%0d %0d %s", s == "hello sv", n.atoi(), s);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
8|HELLO SV|Hell|0
0 42 Hello SV
```

Concatenation gives "Hello SV", 8 characters. `toupper()` returns a new string and does not change `s`. `substr(0, 3)` returns characters 0 to 3 inclusive. `compare` returns 0 for equal strings. String comparison is case-sensitive. `atoi()` converts the leading decimal digits and stops at the first other character.

Tasks and Functions

Q153. Static and automatic variables.

```
module q153;
  function void count_s();
    static int c = 0;
    c++;
    $display("s=%0d", c);
  endfunction

  function automatic void count_a();
    int c = 0;
    c++;
    $display("a=%0d", c);
  endfunction

  initial repeat (3) begin
    count_s();
    count_a();
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
s=1
a=1
s=2
a=1
s=3
a=1
```

The static variable is created and initialised once, before simulation starts, and keeps its value between calls. The automatic variable is created and initialised on every call.

Q154. Passing by value and by reference.

```
module q154;
  function automatic void by_val(int x);         x = x + 10;  endfunction
  function automatic void by_ref(ref int x);     x = x + 10;  endfunction
  function automatic void arr_val(int a [3]);    a[0] = 99;   endfunction
  function automatic void arr_ref(ref int a [3]); a[0] = 99; endfunction

  int v = 1;
  int arr [3] = '{1, 2, 3};

  initial begin
    by_val(v);     $display("%0d", v);
    by_ref(v);     $display("%0d", v);
    arr_val(arr);  $display("%0d", arr[0]);
    arr_ref(arr);  $display("%0d", arr[0]);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
1
11
1
99
```

An argument with no direction is an `input`: the routine works on a copy, for scalars and for whole arrays. With `ref` the routine works on the caller's variable.

Classes

Q155. Handle assignment and copy.

```
module q155;
  class pkt;
    int data;
  endclass

  pkt p1, p2, p3;

  initial begin
    p1 = new();
    p1.data = 5;
    p2 = p1;
    p2.data = 7;
    p3 = new p1;
    p3.data = 9;
    $display("%0d %0d %0d", p1.data, p2.data, p3.data);
    $display("%b %b", p1 == p2, p1 == p3);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
7 7 9
1 0
```

`p2 = p1` copies the handle, so `p1` and `p2` are the same object and the write through `p2` is seen through `p1`. `new p1` creates a second object with a copy of the data. `==` on handles compares whether they refer to the same object.

Q156. Shallow copy with a nested object.

```
module q156;
  class hdr;
    int id;
  endclass

  class pkt;
    int len;
    hdr h;
    function new();
      h = new();
    endfunction
  endclass

  pkt a, b;

  initial begin
    a = new();
    a.len  = 1;
    a.h.id = 10;
    b = new a;
    b.len  = 2;
    b.h.id = 20;
    $display("a: len=%0d id=%0d", a.len, a.h.id);
    $display("b: len=%0d id=%0d", b.len, b.h.id);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
a: len=1 id=20
b: len=2 id=20
```

The shallow copy duplicates `len` but copies only the handle `h`, so both packets share one `hdr` object. Changing `b.h.id` changes what `a.h.id` reads. A deep copy would need `b.h = new a.h;`.

Q157. Virtual and non-virtual methods.

```
module q157;
  class base;
    function void show();          $display("base show");   endfunction
    virtual function void vshow(); $display("base vshow");  endfunction
  endclass

  class child extends base;
    function void show();          $display("child show");  endfunction
    virtual function void vshow(); $display("child vshow"); endfunction
  endclass

  base  b;
  child c;

  initial begin
    c = new();
    b = c;
    b.show();
    b.vshow();
    c.show();
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
base show
child vshow
child show
```

`show` is not virtual, so the type of the handle (`base`) selects the method. `vshow` is virtual, so the type of the object (`child`) selects it. Through the `child` handle the child's `show` is called.

Q158. Constructor order.

```
module q158;
  class A;
    function new();
      $display("A new");
    endfunction
  endclass

  class B extends A;
    int x = 5;
    function new();
      $display("B new x=%0d", x);
      x = 7;
    endfunction
  endclass

  class C extends B;
    function new();
      super.new();
      $display("C new x=%0d", x);
    endfunction
  endclass

  C c;
  initial c = new();
endmodule
```

Answer:

EXPECTED OUTPUT

```
A new
B new x=5
C new x=7
```

Construction runs from the base class down. `B` has no explicit `super.new()`, so the compiler inserts one. The declaration initialiser `x = 5` is applied after `A`'s constructor returns and before the body of `B`'s constructor.

Q159. Static property.

```
module q159;
  class counter;
    static int count;
    int        id;
    function new();
      count++;
      id = count;
    endfunction
  endclass

  counter a, b, c;

  initial begin
    $display("%0d", counter::count);
    a = new();
    b = new();
    c = new a;
    $display("%0d %0d %0d %0d", a.id, b.id, c.id, counter::count);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
0
1 2 1 2
```

The static property exists before any object does. `a` and `b` each run the constructor. `c = new a` is a shallow copy: it does not run the constructor, so `count` stays 2 and `c.id` is a copy of `a.id`.

Q160. Property hiding and `super`.

```
module q160;
  class base;
    int x = 1;
    virtual function int get();
      return x;
    endfunction
  endclass

  class child extends base;
    int x = 2;
    virtual function int get();
      return x + super.get();
    endfunction
  endclass

  base  b;
  child c;

  initial begin
    c = new();
    b = c;
    $display("%0d %0d %0d", b.x, c.x, b.get());
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
1 2 3
```

The child object contains two properties called `x`. Property access depends on the handle type: `b.x` is the base property (1) and `c.x` is the child property (2). `get` is virtual, so `b.get()` runs the child version: its own `x` (2) plus the base method's result (1).

Q161. Downcasting.

```
module q161;
  class base;
  endclass

  class child extends base;
    int extra = 5;
  endclass

  base  b;
  child c;

  initial begin
    b = new();
    if ($cast(c, b)) $display("cast 1 ok");
    else             $display("cast 1 failed, c is null: %b", c == null);

    c = new();
    b = c;
    c = null;
    if ($cast(c, b)) $display("cast 2 ok, extra=%0d", c.extra);
    else             $display("cast 2 failed");
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
cast 1 failed, c is null: 1
cast 2 ok, extra=5
```

In the first case the object is a plain `base`; it has no `extra` property, so it cannot be referred to by a `child` handle and `$cast` returns 0, leaving `c` unchanged. In the second case the object is a `child` referred to by a base handle, so the downcast succeeds.

Threads and Communication

Q162. `join_any`, `join_none` and `wait fork`.

```
module q162;
  initial begin
    fork
      #10 $display("[%0t] A", $time);
      #20 $display("[%0t] B", $time);
    join_any
    $display("[%0t] after join_any", $time);
    fork
      #5 $display("[%0t] C", $time);
    join_none
    $display("[%0t] after join_none", $time);
    wait fork;
    $display("[%0t] done", $time);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
[10] A
[10] after join_any
[10] after join_none
[15] C
[20] B
[20] done
```

`join_any` releases the parent when the first thread (A) finishes at time 10; B keeps running. `join_none` does not block, so the next line prints at time 10. `wait fork` blocks until C (time 15) and B (time 20) have both finished.

Q163. Threads created in a loop.

```
module q163;
  initial begin
    for (int i = 0; i < 3; i++)
      fork
        $display("A: i=%0d", i);
      join_none
    #1;
    for (int i = 0; i < 3; i++)
      fork
        automatic int k = i;
        #(k + 1) $display("[%0t] B: k=%0d", $time, k);
      join_none
    wait fork;
    $display("[%0t] done", $time);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
A: i=3
A: i=3
A: i=3
[2] B: k=0
[3] B: k=1
[4] B: k=2
[4] done
```

The first three threads do not start until the parent blocks at `#1`, by which time the loop has finished and the shared `i` is 3. In the second loop each thread has its own `k`, initialised when the thread is created. The threads are created at time 1 and wait 1, 2 and 3 time units.

Q164. `disable fork`.

```
module q164;
  initial begin
    fork
      begin #10 $display("[%0t] T1 done", $time); end
      begin #30 $display("[%0t] T2 done", $time); end
    join_any
    disable fork;
    #50 $display("[%0t] end", $time);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
[10] T1 done
[60] end
```

T1 finishes at time 10 and releases `join_any`. `disable fork` kills T2, which never prints. The parent then waits 50 more time units.

Q165. A bounded mailbox.

```
module q165;
  mailbox #(int) mbx = new(2);

  initial
    for (int i = 1; i <= 3; i++) begin
      mbx.put(i);
      $display("[%0t] put %0d", $time, i);
    end

  initial begin
    int v;
    #10;
    repeat (3) begin
      mbx.get(v);
      $display("[%0t] got %0d", $time, v);
      #10;
    end
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
[0] put 1
[0] put 2
[10] got 1
[10] put 3
[20] got 2
[30] got 3
```

The mailbox holds two items, so the third `put` blocks at time 0. At time 10 the consumer removes one item and prints; when it then blocks at `#10`, the producer runs, completes its `put` and prints. Items come out in the order they went in.

Q166. `@e` compared with `wait (e.triggered)`.

```
module q166;
  event e;
  initial begin
    #10 -> e;
    fork
      begin @e;                 $display("[%0t] @e returned", $time);   end
      begin wait (e.triggered); $display("[%0t] wait returned", $time); end
    join_any
    $display("[%0t] after join_any", $time);
    #20 $display("[%0t] the @e thread is still blocked", $time);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
[10] wait returned
[10] after join_any
[30] the @e thread is still blocked
```

The event is triggered at time 10 before either thread starts waiting. `e.triggered` stays true until the end of time step 10, so the `wait` passes at once. `@e` waits for a new trigger, which never comes, so that thread blocks for the rest of the simulation.

Q167. A semaphore with one key.

```
module q167;
  semaphore sem = new(1);

  task automatic user(string name, int hold);
    sem.get(1);
    $display("[%0t] %s got the key", $time, name);
    #hold;
    sem.put(1);
  endtask

  initial begin
    fork
      user("A", 10);
      #1 user("B", 5);
    join
    $display("[%0t] both finished", $time);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
[0] A got the key
[10] B got the key
[15] both finished
```

A takes the only key at time 0. B asks at time 1 and blocks until A returns the key at time 10. B holds it for 5 time units, and `join` waits for both.

Assignments and Randomization

Q168. Blocking and nonblocking assignments.

```
module q168;
  logic [3:0] a = 1, b = 2, c = 3, d = 4;
  initial begin
    a = b;
    b = a;
    c <= d;
    d <= c;
    $display("%0d %0d %0d %0d", a, b, c, d);
    #1 $display("%0d %0d %0d %0d", a, b, c, d);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
2 2 3 4
2 2 4 3
```

The blocking assignments take effect immediately: `a` becomes 2, then `b` is assigned the new `a`, also 2. The nonblocking assignments read the old `c` and `d` but have not updated them when the first `$display` runs. After the NBA region, `c` and `d` are swapped.

Q169. Does `randomize()` succeed, and with what values?

```
module q169;
  class pair;
    rand bit [3:0] a, b;
    constraint c { a < 3; b < 3; a > b; a + b == 3; }
  endclass

  pair p;
  int  ok;

  initial begin
    p  = new();
    ok = p.randomize();
    $display("%0d %0d %0d", ok, p.a, p.b);
    ok = p.randomize() with { b == 2; };
    $display("%0d %0d %0d", ok, p.a, p.b);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
1 2 1
0 2 1
```

Both variables must be 0, 1 or 2, `a` must be greater than `b`, and the sum must be 3. The only solution is `a` = 2, `b` = 1, so the first call always produces it. The inline constraint `b == 2` would need `a` > 2, which contradicts `a < 3`: the call returns 0 and the variables keep their previous values. (The simulator also prints a constraint-failure warning.)

Q170. `rand_mode()` and `constraint_mode()`.

```
module q170;
  class cfg;
    rand bit [3:0] x, y;
    constraint lim { x == 5; y == x + 1; }
  endclass

  cfg c;
  int ok;

  initial begin
    c  = new();
    ok = c.randomize();
    $display("%0d %0d %0d", ok, c.x, c.y);

    c.y.rand_mode(0);
    c.y = 9;
    ok  = c.randomize();
    $display("%0d %0d %0d", ok, c.x, c.y);

    c.lim.constraint_mode(0);
    c.x.rand_mode(0);
    ok = c.randomize();
    $display("%0d %0d %0d", ok, c.x, c.y);
  end
endmodule
```

Answer:

EXPECTED OUTPUT

```
1 5 6
0 5 9
1 5 9
```

First call: `x` = 5 forces `y` = 6. Second call: `y` is no longer random, so it is a fixed value 9 that must still satisfy `y == x + 1`; that needs `x` = 8, which contradicts `x == 5`, so the call fails and nothing changes. Third call: the constraint block is off and neither variable is random, so there is nothing to solve and nothing to violate; the call succeeds and the values stay as they are.

> **Interview trap:** Switching off `rand_mode` does not remove a variable from the constraints. It turns the variable into a constant that the constraints must still accept. This is the difference from `constraint_mode(0)`, which removes the constraint.

Code-Debugging Questions

Every fragment in this part compiles. It misbehaves at run time: a crash, a hang, wrong data, or a test that passes when it should not. For each one, find the bug, explain the mechanism, and give the fix. In an interview, say what symptom you would see first; that shows you have debugged real testbenches.

Q171. The simulation stops with a "null object access" error in the driver.

```
class alu_driver;
  mailbox #(alu_txn) gen2drv_mbx;
  virtual alu_if     vif;

  task run();
    alu_txn txn;
    forever begin
      gen2drv_mbx.get(txn);
      @(vif.drv_cb);
      vif.drv_cb.a <= txn.a;
    end
  endtask
endclass
```

Answer: Nothing ever assigns `gen2drv_mbx` or `vif`. Both are handles whose default value is `null`, so the first `gen2drv_mbx.get()` dereferences a null handle. Declaring a mailbox variable does not create a mailbox. Fix: give the driver a constructor that receives both handles from the environment, `function new(mailbox #(alu_txn) mbx, virtual alu_if vif);`, and make the environment construct the mailbox with `new()` and pass the same handle to the generator and the driver. Note that `txn` being null before the `get` is fine: `get` assigns it.

Q172. The generator sends ten transactions, but the driver drives the same values ten times.

```
task run();
  alu_txn txn = new();
  repeat (10) begin
    if (!txn.randomize()) $fatal(1, "randomize failed");
    gen2drv_mbx.put(txn);
  end
  -> gen_done;
endtask
```

Answer: Only one object is created. `put(txn)` stores the handle, not a copy, so the mailbox holds ten handles to the same object. The loop does not block (unbounded mailbox), so all ten randomizations happen before the driver runs, and the driver sees the last values ten times. Fix: construct a new object for every transaction, `txn = new();` inside the loop, or put a copy, `gen2drv_mbx.put(txn.copy());`. The second form is used when a "blueprint" object is kept so that a test can replace it with an extended class.

Q173. The generator reports that it put transactions, but the driver waits forever.

```
class alu_generator;
  mailbox #(alu_txn) gen2drv_mbx = new();
  // ...
endclass

class alu_driver;
  mailbox #(alu_txn) gen2drv_mbx = new();
  // ...
endclass
```

Answer: Each class constructs its own mailbox. The two variables have the same name but refer to two different mailbox objects, so the items the generator puts never reach the driver, which blocks on `get()` of an empty mailbox. Fix: construct one mailbox in the environment and pass that single handle to both components.

Q174. The scoreboard sometimes never sees the end-of-test event.

```
// generator                        // test
task run();                         task run_test();
  // ... put all transactions         fork
  -> gen_done;                          gen.run();
endtask                               join_none
                                      @(gen.gen_done);
                                      $display("generator finished");
                                    endtask
```

Answer: If the generator needs no simulation time (it only fills an unbounded mailbox), it triggers `gen_done` in the same time step in which the test reaches `@(gen.gen_done)`. Which happens first depends on scheduling: with `join_none` the parent normally continues first and reaches the `@`, but any reordering (a `#0`, calling `gen.run()` from another thread) makes the trigger come first, and then `@` waits forever for a trigger that has already happened. Fix: `wait (gen.gen_done.triggered);`, which is true for the rest of the time step in which the event fired, or trigger with the nonblocking `->> gen_done;`.

Q175. After the first timeout check, the monitor stops collecting transactions.

```
task run();
  fork
    mon.run();                      // runs forever
  join_none

  forever begin
    fork
      wait_for_valid_out();
      #1000 $error("timeout waiting for valid_out");
    join_any
    disable fork;
  end
endtask
```

Answer: `disable fork` kills all threads spawned by the calling process, and `mon.run()` was spawned by this same process, so it is killed together with the losing branch of the timeout. Fix: isolate the timeout in its own process so that it has no other children:

```
forever begin
  fork
    begin
      fork
        wait_for_valid_out();
        #1000 $error("timeout waiting for valid_out");
      join_any
      disable fork;
    end
  join
end
```

Q176. The checksum computed by the testbench differs from the DUT's whenever a payload byte is 0x80 or more.

```
byte payload [];
int  checksum;

function void calc();
  checksum = 0;
  foreach (payload[i]) checksum += payload[i];
endfunction
```

Answer: `byte` is signed. A value such as `8'h90` is -112, and it is sign-extended when added to the 32-bit `checksum`, so the sum decreases instead of increasing by 144. Fix: declare the payload as `bit [7:0] payload [];` (or `byte unsigned`). A mismatch that appears only when the top bit of a value is set almost always points to signedness.

Q177. The DUT output is X for the whole test, yet the test reports zero errors.

```
function bit compare(alu_txn exp);
  if (this.result != exp.result) begin
    $error("result mismatch: got %0h, expected %0h", result, exp.result);
    return 0;
  end
  return 1;
endfunction
```

Answer: With `result` equal to X, `!=` evaluates to X, the `if` treats X as false, and the function returns 1 ("match"). Fix: use the case inequality operator, `if (this.result !== exp.result)`, which treats X as a value that differs from every known value. Also make sure `result` is declared `logic [8:0]` in the transaction; if it were `bit`, the X would already have become 0 in the monitor.

Q178. The simulation hangs at time 0. Nothing is driven.

```
// environment:  gen2drv_mbx = new(1);

// generator                          // driver
task run();                           task run();
  repeat (10) begin                     alu_txn txn;
    txn = new();                        @(start_drv);
    void'(txn.randomize());             forever begin
    gen2drv_mbx.put(txn);                 gen2drv_mbx.get(txn);
  end                                     drive(txn);
  -> start_drv;                         end
endtask                               endtask
```

Answer: A deadlock. The mailbox is bounded to one item. The generator's second `put()` blocks until the driver takes the first item. The driver is waiting for `start_drv`, which the generator triggers only after all ten puts. Each is waiting for the other. Fix: remove the start event (the mailbox already synchronises them), or make the mailbox unbounded. In general, draw the "who waits for whom" graph; a cycle is a deadlock.

Q179. Two threads call the same task with different IDs; both messages show the same ID.

```
module tb;
  task send(input int id, input int dly);
    #dly;
    $display("[%0t] sent id %0d", $time, id);
  endtask

  initial fork
    send(1, 10);
    send(2, 20);
  join
endmodule
```

Answer: A task in a module is static by default: there is one copy of `id` and `dly`. Whichever call starts second overwrites `id` while the first is suspended at its delay. Each call has already evaluated its own delay, so the messages still appear at times 10 and 20, but both show the same `id` (2 if the first branch of the fork starts first, which is the usual case). Fix: `task automatic send(...)`.

Q180. The DUT sometimes captures stimulus one cycle early, depending on the simulator.

```
task drive(alu_txn txn);
  @(posedge vif.clk);
  vif.a        = txn.a;
  vif.b        = txn.b;
  vif.op       = txn.op;
  vif.valid_in = 1'b1;
endtask
```

Answer: A race. The driver assigns the DUT inputs with blocking assignments in the Active region of the same clock edge at which the DUT's `always_ff` samples them. The order of the two is undefined, so one simulator captures the new values at this edge and another at the next edge. Fix: drive through the clocking block, `@(vif.drv_cb); vif.drv_cb.a <= txn.a;`, which drives after the design has sampled. Without a clocking block, use nonblocking assignments or drive on the opposite clock edge.

Q181. One test drives the same stimulus as the previous transaction now and then, and coverage is lower than expected.

```
class alu_txn;
  rand bit [7:0] a, b;
  constraint small_c { a < 16; }
endclass

// in the test
txn.randomize() with { a > 100; };
```

Answer: The inline constraint contradicts the class constraint, so `randomize()` fails, returns 0 and leaves `a` and `b` unchanged. The return value is thrown away, so the test carries on with the old values. Fix the contradiction (switch the class constraint off with `txn.small_c.constraint_mode(0);`, or make it `soft`) and always check the result: `if (!txn.randomize() with { a > 100; }) $fatal(1, "randomize failed");`.

Q182. The constraint is meant to produce `lo < mid < hi`, but values such as lo=200, mid=5, hi=9 appear.

```
rand bit [7:0] lo, mid, hi;
constraint order_c { lo < mid < hi; }
```

Answer: Relational operators cannot be chained. The expression is parsed as `(lo < mid) < hi`: the 1-bit result of `lo < mid` (0 or 1) is compared with `hi`. Any `hi` of 2 or more satisfies it regardless of `lo` and `mid`. Fix: `constraint order_c { lo < mid; mid < hi; }`.

Q183. The scoreboard's queue of observed transactions contains N copies of the last transaction.

```
class alu_monitor;
  mailbox #(alu_txn) mon2scb_mbx;
  virtual alu_if     vif;
  alu_txn            txn = new();

  task run();
    forever begin
      @(vif.mon_cb);
      if (vif.mon_cb.valid_out) begin
        txn.result = vif.mon_cb.result;
        mon2scb_mbx.put(txn);
      end
    end
  endtask
endclass
```

Answer: The same mistake as Q172, on the monitor side. One object is reused, the mailbox and the scoreboard's queue hold handles to it, and every new sample overwrites the fields that all those handles see. It may appear to work if the scoreboard consumes each item before the next sample, and fail as soon as it stores them. Fix: `txn = new();` for each observed transaction.

Q184. Every comparison fails: the observed result always belongs to the previous operation.

```
// monitor
forever begin
  @(vif.mon_cb);
  if (vif.mon_cb.valid_in) begin
    txn = new();
    txn.a      = vif.mon_cb.a;
    txn.b      = vif.mon_cb.b;
    txn.op     = vif.mon_cb.op;
    txn.result = vif.mon_cb.result;
    mon2scb_mbx.put(txn);
  end
end
```

Answer: The ALU registers its result: `result` and `valid_out` appear one clock after `valid_in`. The monitor samples `result` in the same cycle as the inputs, so it pairs each set of inputs with the result of the operation before it. Fix: capture the inputs when `valid_in` is 1, and capture `result` when `valid_out` is 1 on a later cycle, then send the transaction. A constant one-cycle offset between expected and actual is the signature of sampling at the wrong pipeline stage.

Q185. The simulation stops advancing: the time stays at 0 and the simulator uses 100% CPU.

```
task run();
  alu_txn txn;
  forever begin
    if (mon2scb_mbx.num() > 0) begin
      mon2scb_mbx.get(txn);
      check(txn);
    end
  end
endtask
```

Answer: When the mailbox is empty the loop body contains no blocking statement, so the `forever` loop spins without ever letting simulation time advance. No other process can run, so the mailbox never fills. Fix: remove the polling and let `get()` block: `forever begin mon2scb_mbx.get(txn); check(txn); end`. Every `forever` loop must contain a statement that is guaranteed to block on each iteration.

Q186. A `randc` field repeats values long before all values have been used.

```
class id_txn;
  randc bit [3:0] id;
endclass

repeat (16) begin
  id_txn t = new();
  void'(t.randomize());
  $display("%0d", t.id);
end
```

Answer: The cyclic history of a `randc` variable is stored in the object. A new object is constructed in every iteration, so every call is the first call of a new cycle and the values are effectively plain random. Fix: construct the object once and call `randomize()` on it sixteen times; if separate objects are needed downstream, send copies.

Find-the-Error Questions

Each fragment is rejected by the compiler or the elaborator. Name the rule that is broken and correct the code.

Q187.

```
function int wait_and_add(int a, int b);
  #10;
  return a + b;
endfunction
```

Answer: A function cannot contain a delay or any other time-consuming statement. Make it a task with an `output` argument, or remove the delay.

Q188.

```
class alu_txn;
  int        id;
  static int count;
  static function int next_id();
    id = count++;
    return id;
  endfunction
endclass
```

Answer: A static method has no object (`this`), so it cannot access the non-static property `id`. Either make the method non-static or have it use only `count`: `return count++;`.

Q189.

```
class cfg;
  local int secret = 42;
endclass

module tb;
  cfg c = new();
  initial $display("%0d", c.secret);
endmodule
```

Answer: A `local` member is accessible only inside the methods of its own class. Add a public accessor, `function int get_secret(); return secret; endfunction`, or remove `local`.

Q190.

```
class base;                endclass
class child extends base;  endclass

module tb;
  base  b = new();
  child c;
  initial c = b;
endmodule
```

Answer: A base-class handle cannot be assigned to a derived-class handle directly. Use `$cast(c, b)`, which compiles and checks at run time. (Here the cast would fail, because the object is a plain `base`.)

Q191.

```
virtual class shape;
  pure virtual function real area();
endclass

module tb;
  shape s = new();
endmodule
```

Answer: An abstract (`virtual`) class cannot be constructed. Declare a derived class that implements `area()` and construct that; the handle may still be of type `shape`.

Q192.

```
virtual class shape;
  pure virtual function real area();
endclass

class square extends shape;
  real side;
endclass
```

Answer: `square` is not abstract but does not implement the pure virtual method `area()`. Implement it, `virtual function real area(); return side * side; endfunction`, or declare `square` as `virtual class`.

Q193.

```
class pkt;
  randc bit [2:0] kind;
  constraint kind_c { kind dist {0 := 50, [1:7] := 10}; }
endclass
```

Answer: `dist` cannot be applied to a `randc` variable; a weighted distribution contradicts cyclic behaviour. Declare `kind` as `rand`, or drop the `dist`.

Q194.

```
class child extends base;
  int n;
  function new(int n);
    this.n = n;
    super.new();
  endfunction
endclass
```

Answer: A call to `super.new()` must be the first statement of the constructor. Move it above `this.n = n;`.

Q195.

```
class alu_driver;
  alu_if vif;
endclass
```

Answer: A class cannot contain an interface instance. Declare a virtual interface, `virtual alu_if vif;`, and assign the real instance to it from the testbench top.

Q196.

```
class pkt;
  rand bit       mode;
  rand bit [3:0] len;
  constraint len_c { if (mode) len = 4; }
endclass
```

Answer: A constraint is a set of Boolean expressions, not procedural code, so an assignment is not allowed. Use equality: `if (mode) len == 4;`.

Q197.

```
int d [];
initial begin
  d.push_back(5);
end
```

Answer: `push_back` is a queue method; a dynamic array does not have it. Declare a queue, `int d [$];`, or resize the dynamic array with `d = new[d.size() + 1](d);` and assign the last element.

Q198.

```
task automatic wait_clk();
  @(posedge clk);
endtask

function automatic void sample();
  wait_clk();
endfunction
```

Answer: A function cannot call a task, because a task may consume time. Make `sample` a task.

Q199.

```
int m [2][3];
initial
  foreach (m[i][j]) m[i][j] = i + j;
```

Answer: The loop variables of a multidimensional `foreach` are written inside one pair of brackets, separated by commas: `foreach (m[i, j]) m[i][j] = i + j;`.

Q200.

```
module tb;
  task incr(ref int x);
    x++;
  endtask
endmodule
```

Answer: A `ref` argument is allowed only in a task or function with automatic lifetime, and tasks in a module are static by default. Write `task automatic incr(ref int x);`.

Q201.

```
class alu_monitor;
  virtual alu_if vif;
  a_valid: assert property (@(posedge vif.clk) vif.valid_in |=> vif.valid_out);
endclass
```

Answer: Concurrent assertions are not allowed inside a class; they must be in a module, interface, program or checker. Put the assertion in `alu_if` (or in a module attached with `bind`). Immediate assertions are allowed in class methods.

Q202.

```
interface alu_if (input logic clk);
  logic [7:0] a;
  logic [8:0] result;
  modport MON (input a, result);
endinterface

module mon_block (alu_if.MON bus);
  assign bus.result = 9'd0;
endmodule
```

Answer: Through the `MON` modport, `result` is an input, so this module is not allowed to drive it. Either the module should not drive it (a monitor never drives), or it needs a modport in which `result` is an output.

Write-the-Code Questions

These are the short programs interviewers ask you to write in a shared editor. Aim to write each one in under ten minutes. State your assumptions aloud, write the simplest correct version first, then mention improvements.

Arrays and Basic Logic

Q203. Reverse a queue without using `reverse()`.

```
module q203;
  int q [$] = '{1, 2, 3, 4, 5};
  int r [$];
  int tmp;

  initial begin
    // Method 1: in place, swapping from both ends
    for (int i = 0, j = q.size() - 1; i < j; i++, j--) begin
      tmp  = q[i];
      q[i] = q[j];
      q[j] = tmp;
    end
    $display("%p", q);                    // 5 4 3 2 1

    // Method 2: pop from the back of one queue, push to the back of another
    while (q.size() > 0) r.push_back(q.pop_back());
    $display("%p", r);                    // 1 2 3 4 5 again
  end
endmodule
```

Points to mention: a `for` loop may declare and update two variables; method 2 empties the source queue.

Q204. Remove duplicate values from a queue, keeping the first occurrence of each and the original order.

```
module q204;
  int q [$] = '{3, 7, 3, 1, 7, 9, 1};
  int r [$];
  bit seen [int];                         // associative array used as a set

  initial begin
    foreach (q[i])
      if (!seen.exists(q[i])) begin
        seen[q[i]] = 1;
        r.push_back(q[i]);
      end
    $display("%p", r);                    // 3 7 1 9
  end
endmodule
```

`q.unique()` also returns the distinct values, but the standard does not promise that it keeps the original order, so the explicit loop is the safe answer. The associative array makes the membership test fast.

Q205. Find the second largest distinct value in an array.

```
module q205;
  int a [] = '{4, 9, 2, 9, 7};
  int big, second;
  bit found;

  initial begin
    big = a[0];
    foreach (a[i]) begin
      if (a[i] > big) begin
        second = big;
        big    = a[i];
        found  = 1;
      end
      else if (a[i] != big && (!found || a[i] > second)) begin
        second = a[i];
        found  = 1;
      end
    end
    if (found) $display("largest=%0d second=%0d", big, second);   // 9 and 7
    else       $display("all elements are equal");
  end
endmodule
```

One pass, no sorting. The `found` flag handles an array whose elements are all equal. The shorter alternative, acceptable if speed does not matter: `r = a.unique(); r.rsort();` and take `r[1]`.

Q206. Count how many times each value occurs in an array.

```
module q206;
  int data [] = '{5, 3, 5, 8, 3, 5};
  int count [int];                        // key = value, entry = number of occurrences

  initial begin
    foreach (data[i])
      if (count.exists(data[i])) count[data[i]]++;
      else                       count[data[i]] = 1;
    foreach (count[k]) $display("%0d occurs %0d times", k, count[k]);
  end
endmodule
```

EXPECTED OUTPUT

```
3 occurs 2 times
5 occurs 3 times
8 occurs 1 times
```

The associative array is iterated in ascending key order. Many engineers write just `count[data[i]]++;`, relying on a new entry starting at 0; the explicit `exists` test avoids any warning about reading an entry that does not exist.

Q207. Swap two variables without a temporary variable.

```
module q207;
  logic [7:0] a = 8'h3C, b = 8'hA5;
  initial begin
    {a, b} = {b, a};                    // 1. concatenation: RHS is evaluated first
    a = a ^ b;  b = a ^ b;  a = a ^ b;  // 2. XOR swap (swaps back)
    a = a + b;  b = a - b;  a = a - b;  // 3. arithmetic swap (wraps correctly)
    $display("%h %h", a, b);            // a5 3c after three swaps
    a <= b;  b <= a;                    // 4. nonblocking assignments
    #1 $display("%h %h", a, b);         // 3c a5
  end
endmodule
```

Method 1 is the idiomatic answer. Method 4 is how two registers are swapped in RTL.

Q208. Write a clock generator with a parameterised frequency and duty cycle.

```
`timescale 1ns/1ps

module clk_gen #(
  parameter real FREQ_MHZ = 100.0,
  parameter real DUTY_PCT = 40.0
) (
  output logic clk
);
  real period_ns, t_high, t_low;

  initial begin
    period_ns = 1000.0 / FREQ_MHZ;           // 100 MHz -> 10 ns
    t_high    = period_ns * DUTY_PCT / 100.0; // 4 ns high
    t_low     = period_ns - t_high;           // 6 ns low
    clk = 1'b0;
    forever begin
      #t_low  clk = 1'b1;
      #t_high clk = 1'b0;
    end
  end
endmodule
```

Mention that the time precision (1 ps here) limits the accuracy: a period that is not a multiple of the precision is rounded.

Q209. Write a detector for the serial bit sequence 1011, with overlapping sequences allowed.

```
module seq_det_1011 (
  input  logic clk, rst_n, din,
  output logic det
);
  typedef enum logic [1:0] {S0, S1, S10, S101} state_e;
  state_e state, next_state;

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) state <= S0;
    else        state <= next_state;

  always_comb begin
    next_state = S0;
    unique case (state)
      S0  : next_state = din ? S1   : S0;
      S1  : next_state = din ? S1   : S10;
      S10 : next_state = din ? S101 : S0;
      S101: next_state = din ? S1   : S10;   // overlap: the last 1 starts a new match
    endcase
  end

  assign det = (state == S101) && din;        // Mealy output
endmodule
```

Each state name records how much of the pattern has been seen. From `S101`, an input of 1 completes the pattern and leaves "1" as the start of the next one; an input of 0 leaves "10". For the stream 1011011, `det` is 1 at the fourth and the seventh bit. Without overlap, `S101` with `din` = 1 would return to `S0`.

Q210. Write a circuit that detects rising and falling edges of a signal.

```
module edge_det (
  input  logic clk, rst_n, sig,
  output logic rise, fall, any_edge
);
  logic sig_q;                                // sig delayed by one clock

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) sig_q <= 1'b0;
    else        sig_q <= sig;

  assign rise     =  sig & ~sig_q;            // 0 -> 1
  assign fall     = ~sig &  sig_q;            // 1 -> 0
  assign any_edge =  sig ^  sig_q;
endmodule
```

Each output is a one-cycle pulse. If `sig` comes from another clock domain it must first pass through a two-flop synchroniser.

Q211. Write recursive factorial and an iterative check for a palindrome string.

```
module q211;
  function automatic longint unsigned fact(int unsigned n);
    return (n <= 1) ? 1 : n * fact(n - 1);
  endfunction

  function automatic bit is_palindrome(string s);
    for (int i = 0, j = s.len() - 1; i < j; i++, j--)
      if (s[i] != s[j]) return 0;
    return 1;
  endfunction

  initial begin
    $display("%0d %0d", fact(5), fact(0));                      // 120 1
    $display("%b %b", is_palindrome("level"), is_palindrome("sv")); // 1 0
  end
endmodule
```

The function must be `automatic` to be recursive in a module: a static function has a single copy of `n`, which the nested calls would overwrite.

Classes

Q212. Write a singleton class.

```
class tb_config;
  local static tb_config inst;            // the one object
  int unsigned num_txn = 10;

  local function new();                   // nobody outside the class can construct it
  endfunction

  static function tb_config get();
    if (inst == null) inst = new();
    return inst;
  endfunction
endclass

module q212;
  tb_config c1, c2;
  initial begin
    c1 = tb_config::get();
    c2 = tb_config::get();
    c1.num_txn = 50;
    $display("%0d %b", c2.num_txn, c1 == c2);      // 50 1
  end
endmodule
```

A singleton guarantees that exactly one object of the class exists and gives global access to it. The three ingredients are a static handle, a `local` constructor and a static `get()` function. It is used for a shared configuration or report object.

Q213. Write a class with a nested object and a correct deep copy.

```
class pkt_hdr;
  bit [7:0] src, dst;
  function pkt_hdr copy();
    pkt_hdr c = new();
    c.src = src;
    c.dst = dst;
    return c;
  endfunction
endclass

class packet;
  pkt_hdr   hdr;
  bit [7:0] payload [];

  function new();
    hdr = new();
  endfunction

  virtual function packet copy();
    packet c  = new();
    c.hdr     = hdr.copy();              // nested object: copy it, not the handle
    c.payload = payload;                 // dynamic array: assignment copies the elements
    return c;
  endfunction
endclass
```

Say explicitly which members need care: handles do (they would be shared), arrays of plain values do not (array assignment copies the data), arrays of handles do (each element must be copied).

Q214. Write a parameterised stack class.

```
class stack #(type T = int, int DEPTH = 8);
  local T items [$];

  function bit push(T item);
    if (items.size() >= DEPTH) return 0;          // full
    items.push_back(item);
    return 1;
  endfunction

  function bit pop(output T item);
    if (items.size() == 0) return 0;              // empty
    item = items.pop_back();
    return 1;
  endfunction

  function int size();
    return items.size();
  endfunction
endclass

module q214;
  stack #(string, 2) s = new();
  string str;
  bit    ok_a, ok_b, ok_c;
  initial begin
    ok_a = s.push("a");
    ok_b = s.push("b");
    ok_c = s.push("c");                           // rejected: the stack is full
    $display("%b %b %b", ok_a, ok_b, ok_c);       // 1 1 0
    void'(s.pop(str));
    $display("%s %0d", str, s.size());            // b 1
  end
endmodule
```

Last in, first out: push and pop use the same end of the queue. Returning a status bit instead of printing an error lets the caller decide what a full or empty stack means.

Threads and Communication

Q215. Write a generator and a driver that communicate through a mailbox, where the generator waits for the driver to finish each item.

```
class generator;
  mailbox #(int) gen2drv_mbx;
  event          drv_done;
  int            n;

  function new(mailbox #(int) mbx, event drv_done, int n);
    this.gen2drv_mbx = mbx;
    this.drv_done    = drv_done;
    this.n           = n;
  endfunction

  task run();
    for (int i = 1; i <= n; i++) begin
      gen2drv_mbx.put(i);
      $display("[%0t] gen: sent %0d", $time, i);
      @drv_done;                          // wait until the driver has finished this item
    end
  endtask
endclass

class driver;
  mailbox #(int) gen2drv_mbx;
  event          drv_done;

  function new(mailbox #(int) mbx, event drv_done);
    this.gen2drv_mbx = mbx;
    this.drv_done    = drv_done;
  endfunction

  task run();
    int item;
    forever begin
      gen2drv_mbx.get(item);
      #10;                                // time taken to drive the item
      $display("[%0t] drv: drove %0d", $time, item);
      -> drv_done;
    end
  endtask
endclass

module q215;
  mailbox #(int) mbx = new();
  event          done;
  generator      gen;
  driver         drv;

  initial begin
    gen = new(mbx, done, 2);
    drv = new(mbx, done);
    fork
      gen.run();
      drv.run();
    join_any                              // the generator ends; the driver runs forever
    $display("[%0t] test finished", $time);
    $finish;
  end
endmodule
```

EXPECTED OUTPUT

```
[0] gen: sent 1
[10] drv: drove 1
[10] gen: sent 2
[20] drv: drove 2
[20] test finished
```

The mailbox and the event are created once in the top and the same handles are given to both objects. `@drv_done` is safe here because the driver triggers it 10 time units after the generator started waiting; if the driver could respond in zero time, `wait (drv_done.triggered)` would be needed.

Q216. Two threads must write to the same bus. Protect the bus with a semaphore.

```
module q216;
  semaphore bus_key = new(1);

  task automatic bus_write(string who, int addr);
    bus_key.get(1);                              // wait for exclusive access
    $display("[%0t] %s starts write to %0h", $time, who, addr);
    #10;                                         // the transfer takes time
    $display("[%0t] %s ends write", $time, who);
    bus_key.put(1);                              // always give the key back
  endtask

  initial begin
    fork
      begin bus_write("seq_a", 'h10); #1; bus_write("seq_a", 'h14); end
      #1 bus_write("seq_b", 'h20);
    join
  end
endmodule
```

EXPECTED OUTPUT

```
[0] seq_a starts write to 10
[10] seq_a ends write
[10] seq_b starts write to 20
[20] seq_b ends write
[20] seq_a starts write to 14
[30] seq_a ends write
```

The transfers never overlap. `seq_b` has been waiting since time 1 and receives the key when `seq_a` returns it at time 10; `seq_a` asks again at time 11 and has to wait until time 20. Without the semaphore, the writes of the two threads would overlap from time 1 to time 10. (The `#1` between the two writes of `seq_a` is deliberate: if a thread returns the key and asks for it again in the same instant, simulators differ on whether it or the waiting thread gets the key.)

Q217. Write a task that waits for a signal with a timeout.

```
module q217;
  logic done = 0;

  task automatic wait_done(input time timeout, output bit ok);
    ok = 0;
    fork
      begin                                 // isolating process for disable fork
        fork
          begin @(posedge done); ok = 1; end
          #timeout;
        join_any
        disable fork;                       // kill whichever branch is still waiting
      end
    join
  endtask

  initial begin
    bit ok;
    fork
      #30 done = 1;
    join_none
    wait_done(100, ok);  $display("[%0t] ok=%b", $time, ok);   // [30] ok=1
    done = 0;
    wait_done(100, ok);  $display("[%0t] ok=%b", $time, ok);   // [130] ok=0
  end
endmodule
```

The pattern is `fork` two branches (the awaited condition and the delay), `join_any`, then `disable fork`. The outer `fork`…`join` with one `begin`…`end` makes sure `disable fork` cannot kill unrelated threads of the caller (here, the thread that sets `done`).

Q218. Write a scoreboard that compares observed transactions with expected ones, in order.

```
class simple_scb;
  logic [8:0] exp_q [$];                    // expected results, oldest first
  int unsigned pass_cnt, fail_cnt;

  function void write_expected(logic [8:0] value);
    exp_q.push_back(value);
  endfunction

  function void write_actual(logic [8:0] value);
    logic [8:0] exp;
    if (exp_q.size() == 0) begin
      $error("unexpected output %0h: nothing was expected", value);
      fail_cnt++;
      return;
    end
    exp = exp_q.pop_front();
    if (value !== exp) begin
      $error("mismatch: actual=%0h expected=%0h", value, exp);
      fail_cnt++;
    end
    else pass_cnt++;
  endfunction

  function void report();
    if (exp_q.size() != 0) begin
      $error("%0d expected results never arrived", exp_q.size());
      fail_cnt += exp_q.size();
    end
    $display("scoreboard: %0d passed, %0d failed", pass_cnt, fail_cnt);
  endfunction
endclass
```

Three checks make a scoreboard complete, and interviewers look for all of them: the value comparison (with `!==`), an output that was not expected (empty queue), and expected items that never came out (non-empty queue at the end of the test).

Q219. Start N parallel threads, each with its own ID, and wait until all have finished.

```
module q219;
  task automatic worker(int id);
    #(id * 10) $display("[%0t] worker %0d done", $time, id);
  endtask

  initial begin
    for (int i = 1; i <= 3; i++) begin
      fork
        automatic int id = i;             // one copy per thread
        worker(id);
      join_none
    end
    wait fork;                            // block until all three have finished
    $display("[%0t] all workers done", $time);
  end
endmodule
```

EXPECTED OUTPUT

```
[10] worker 1 done
[20] worker 2 done
[30] worker 3 done
[30] all workers done
```

`fork`...`join` cannot be used because the number of threads is decided by a loop; the combination `join_none` in a loop plus `wait fork` is the standard answer. The `automatic` copy is essential (see Q163).

Constraint-Writing Questions

Constraint writing is the most common hands-on question in a DV interview. The interviewer describes a set of legal values in words and you write the constraint. Three habits earn marks:

- Declare the variable with the right width and state it ("I assume `x` is an 8-bit unsigned value").
- Think about expression width and overflow before writing any arithmetic.
- After writing, test the constraint mentally with one legal and one illegal value.

Each solution below is a complete class, followed by a one-line explanation.

Single Variables

Q220. A value between 10 and 50, but not between 20 and 30.

```
class c220;
  rand bit [7:0] x;
  constraint x_c {
    x inside {[10:50]};
    !(x inside {[20:30]});
  }
endclass
```

Two constraints are ANDed. Equivalent in one line: `x inside {[10:19], [31:50]};`.

Q221. An even number; then a multiple of 4.

```
class c221;
  rand bit [7:0] e, m;
  constraint even_c { e % 2 == 0; }       // or  e[0] == 0
  constraint mul4_c { m[1:0] == 2'b00; }  // or  m % 4 == 0
endclass
```

A bit-select is cheaper for the solver than `%` and says the same thing for powers of two.

Q222. A length that is small (1 to 8) 70% of the time and large (9 to 64) 30% of the time.

```
class c222;
  rand bit [6:0] len;
  constraint len_c { len dist { [1:8] :/ 70, [9:64] :/ 30 }; }
endclass
```

`:/` divides each weight over its range, so the two ranges have total weights 70 and 30. With `:=` each of the 56 values in the second range would have weight 30 and the large values would dominate.

Q223. ALU opcodes: ADD and SUB 40% each, the six other opcodes sharing the remaining 20%.

```
class c223;
  rand alu_op_e op;
  constraint op_c {
    op dist { ALU_ADD := 40, ALU_SUB := 40, [ALU_AND:ALU_SHR] :/ 20 };
  }
endclass
```

The enum values 2 to 7 are covered by one range, and `:/` splits the weight 20 among them.

Q224. Exactly one bit set in a 16-bit value (one-hot).

```
class c224;
  rand bit [15:0] x;
  constraint onehot_c { $onehot(x); }     // or  $countones(x) == 1
endclass
```

Without system functions: `x != 0; (x & (x - 1)) == 0;`. The same constraint answers "generate a power of two".

Q225. Exactly five bits set in a 16-bit value.

```
class c225;
  rand bit [15:0] x;
  constraint five_c { $countones(x) == 5; }
endclass
```

`$countones` is allowed in constraints. A loop-based alternative sums the bits: `x[0] + x[1] + ...` written with care for width (each bit must be widened before adding).

Q226. No two adjacent bits set in an 8-bit value.

```
class c226;
  rand bit [7:0] x;
  constraint no_adj_c { (x & (x >> 1)) == 0; }
endclass
```

Shifting by one lines each bit up with its neighbour; if the AND is zero no pair of neighbours is both 1. Add `$countones(x) == 3` to ask for exactly three non-adjacent ones.

Q227. The upper nibble of a byte is the bitwise inverse of the lower nibble.

```
class c227;
  rand bit [7:0] x;
  constraint inv_c { x[7:4] == ~x[3:0]; }
endclass
```

Both sides are 4 bits wide, so the inversion does not introduce extra upper bits. Had the right side been compared with a wider value, `~` would have been applied after extension, giving wrong upper bits.

Q228. A 32-bit address that is word-aligned, lies in the range 0x1000 to 0x1FFF, and for which a burst of `len` bytes does not cross the end of the range.

```
class c228;
  rand bit [31:0] addr;
  rand bit [7:0]  len;
  constraint addr_c {
    addr[1:0] == 2'b00;
    addr inside {[32'h1000:32'h1FFF]};
    len inside {[1:64]};
    addr + len <= 32'h2000;
  }
endclass
```

`addr + len` is evaluated in 32 bits; because `addr` is at most 0x1FFC and `len` at most 64, the sum cannot overflow. State that reasoning; it is what the interviewer is checking.

Q229. A value that is different from the value generated by the previous call.

```
class c229;
  rand bit [3:0] x;
  bit      [3:0] prev;                    // state variable, not random
  constraint diff_c { x != prev; }
  function void post_randomize();
    prev = x;
  endfunction
endclass
```

`prev` is a non-random variable, so the solver treats it as a constant; `post_randomize()` updates it after every successful call. (The first call can never return 0, the initial value of `prev`; initialise `prev` differently if that matters.)

Q230. Every value of a 3-bit variable exactly once before any repeats, without using `randc`.

```
class c230;
  rand bit [2:0] x;
  bit      [2:0] used [$];
  constraint new_c { !(x inside {used}); }
  function void post_randomize();
    used.push_back(x);
    if (used.size() == 8) used.delete();  // all eight seen: start a new cycle
  endfunction
endclass
```

The queue records the values already used; `inside` accepts an array as its set, and an empty array matches nothing. This is how cyclic behaviour is built when `randc` is not usable, for example when the cycle must cover a pair of variables.

Relations Between Variables

Q231. A packet kind decides the legal payload length: SHORT 1 to 8, MEDIUM 9 to 63, LONG 64 to 255.

```
class c231;
  typedef enum {SHORT, MEDIUM, LONG} kind_e;
  rand kind_e    kind;
  rand bit [7:0] len;
  constraint len_c {
    (kind == SHORT)  -> len inside {[1:8]};
    (kind == MEDIUM) -> len inside {[9:63]};
    (kind == LONG)   -> len inside {[64:255]};
  }
  constraint order_c { solve kind before len; }
endclass
```

Three implications cover the three kinds. Without `solve kind before len`, `LONG` would be chosen about 75% of the time because it has 192 of the 255 (kind, len) solutions; with it, each kind is equally likely.

Q232. When `flag` is 1, `data` must be 0; `flag` should be 1 about half the time.

```
class c232;
  rand bit       flag;
  rand bit [7:0] data;
  constraint data_c { flag -> data == 0; }
  constraint ord_c  { solve flag before data; }
endclass
```

The implication alone gives 257 solutions, only one with `flag` = 1. The ordering makes the solver pick `flag` first, 0 or 1 with equal probability, and then a legal `data`.

Q233. Two 8-bit values whose sum does not overflow 8 bits.

```
class c233;
  rand bit [7:0] a, b;
  constraint sum_c { a + b <= 255; }
endclass
```

This is correct because the literal `255` is a 32-bit integer, so `a` and `b` are extended to 32 bits before the addition. The trap is writing `a + b <= 8'd255`: every operand is then 8 bits, the sum wraps, and the constraint is always true. If in doubt, write `int'(a) + int'(b) <= 255`.

Q234. Three values in strictly increasing order.

```
class c234;
  rand bit [7:0] lo, mid, hi;
  constraint order_c { lo < mid; mid < hi; }
endclass
```

Two separate relations. `lo < mid < hi` compiles but means `(lo < mid) < hi`, which is a different and almost useless constraint (see Q182).

Q235. A valid calendar date (ignoring leap years).

```
class c235;
  rand int unsigned month, day;
  constraint date_c {
    month inside {[1:12]};
    day   inside {[1:31]};
    (month inside {4, 6, 9, 11}) -> day <= 30;
    (month == 2)                 -> day <= 28;
  }
endclass
```

A general range plus implications for the shorter months. To include leap years add a `rand bit leap;` (or a year with the divisibility rule) and write `(month == 2) -> day <= (leap ? 29 : 28);`.

Arrays

Q236. A dynamic array of 5 to 10 elements in strictly ascending order.

```
class c236;
  rand bit [7:0] a [];
  constraint size_c { a.size() inside {[5:10]}; }
  constraint asc_c  { foreach (a[i]) if (i > 0) a[i] > a[i-1]; }
endclass
```

The guard `i > 0` keeps the index `i-1` in range. Using `>=` instead allows equal neighbours. An alternative is to randomize freely and call `a.sort()` in `post_randomize()`.

Q237. An array with all elements different, without the `unique` keyword.

```
class c237;
  rand bit [3:0] a [8];
  constraint uniq_c {
    foreach (a[i])
      foreach (a[j])
        if (i < j) a[i] != a[j];
  }
endclass
```

Every pair is compared once (`i < j` avoids comparing an element with itself and halves the number of constraints). With the keyword: `constraint uniq_c { unique {a}; }`.

Q238. An array of five elements, each between 1 and 40, whose sum is exactly 100.

```
class c238;
  rand bit [7:0] a [5];
  constraint elem_c { foreach (a[i]) a[i] inside {[1:40]}; }
  constraint sum_c  { a.sum() with (int'(item)) == 100; }
endclass
```

`sum()` has the width of an element. Without the cast the sum would be computed modulo 256, and values such as 356 would also satisfy the constraint (not reachable here because of the limit of 40, but the cast is the correct habit and the point the interviewer wants to hear).

Q239. A palindrome array: it reads the same forwards and backwards.

```
class c239;
  rand bit [3:0] a [7];
  constraint pal_c {
    foreach (a[i])
      if (i < $size(a) / 2) a[i] == a[$size(a) - 1 - i];
  }
endclass
```

Each element in the first half must equal its mirror element. The middle element of an odd-length array is unconstrained.

Q240. Fixed patterns: (a) 0, 1, 0, 1, ... and (b) 0, 0, 1, 1, 2, 2, ...

```
class c240;
  rand bit [3:0] alt [8];
  rand bit [3:0] pair [8];
  constraint alt_c  { foreach (alt[i])  alt[i]  == i % 2; }
  constraint pair_c { foreach (pair[i]) pair[i] == i / 2; }
endclass
```

When the interviewer asks for a pattern, look for a formula in terms of the index. Other favourites: `a[i] == i * i`, `a[i] == (i % 2 ? i : 0)`, and the Fibonacci relation `if (i > 1) a[i] == a[i-1] + a[i-2]`.

Q241. A 4 by 4 identity matrix; then a matrix in which each row has exactly one 1.

```
class c241;
  rand bit ident [4][4];
  rand bit onehot_rows [4][4];
  constraint ident_c { foreach (ident[i, j]) ident[i][j] == (i == j); }
  constraint rows_c  {
    foreach (onehot_rows[i]) onehot_rows[i].sum() with (int'(item)) == 1;
  }
endclass
```

A two-dimensional `foreach` uses `[i, j]`. For the second constraint, `onehot_rows[i]` is one row, and the reduction method is applied to it with a cast because each element is one bit.

Q242. A sequence in which each element differs from the previous one in exactly one bit (Gray-code style).

```
class c242;
  rand bit [3:0] a [8];
  constraint gray_c {
    foreach (a[i])
      if (i > 0) $countones(a[i] ^ a[i-1]) == 1;
  }
endclass
```

XOR marks the bits that differ between neighbours; `$countones` requires exactly one such bit.

Q243. Four memory regions, each with a base address and a length, that do not overlap.

```
class c243;
  rand bit [15:0] base [4];
  rand bit [15:0] len  [4];
  constraint lim_c {
    foreach (base[i]) {
      base[i] <= 16'hF000;
      len[i] inside {[1:256]};
    }
  }
  constraint no_overlap_c {
    foreach (base[i])
      foreach (base[j])
        if (i < j)
          (base[i] + len[i] <= base[j]) || (base[j] + len[j] <= base[i]);
  }
endclass
```

Two regions do not overlap if one ends at or before the start of the other. The limits in `lim_c` guarantee that `base + len` cannot exceed 16 bits; without them the 16-bit sum could wrap and a region near the top of memory would appear to end near address 0.

> **Interview trap:** Constraint expressions follow the same width and sign rules as any other expression (M1). Most wrong constraints written in interviews are wrong because of an overflow in a sum, a signed variable (`int`) that goes negative, or a chained comparison. Check those three things every time.

> **Key point:** If you cannot express a requirement cleanly in constraints, say so and use `post_randomize()`: randomize the easy part, then compute, sort or shuffle the rest procedurally. Knowing when not to fight the solver is a sign of experience.

OOP, IPC and Testbench Architecture Questions

These three groups go deeper into the areas where DV interviews spend most of their time. The questions are the "why" and "what if" follow-ups that come after the definitions in the basic and intermediate parts.

OOP-Based Questions

Q244. Explain the four pillars of OOP using a SystemVerilog testbench. *Encapsulation*: data and the methods that work on it are kept together in a class, and access is controlled with `local` and `protected`; the transaction's fields and its `print`, `copy` and `compare` methods are one unit. *Inheritance*: `class err_txn extends alu_txn` reuses everything in the base transaction and adds error-injection fields or constraints. *Polymorphism*: the generator holds an `alu_txn` handle; if the test puts an `err_txn` object there, calls to virtual methods such as `randomize()` and `print()` act on the extended class without any change to the generator. *Abstraction*: a `virtual class` with pure virtual methods defines what every driver must provide (`run`, `drive`) without saying how.

Q245. A test extends the transaction and overrides `print()`, but the log still shows the base-class format. Why? The component calls `txn.print()` through a base-class handle and `print` was not declared `virtual` in the base class, so the handle type selects the base method. Declaring `virtual function void print();` in the base class fixes it; no change is needed in the extended class or in the caller.

Q246. A derived class adds a method that the base class does not have. Can it be called through a base-class handle? No. The compiler only knows the members of the handle's type, so `b.new_method()` is a compile error even if the object is of the derived class. Downcast first: `if ($cast(d, b)) d.new_method();`. If every derived class needs the method, declare it (possibly as pure virtual) in the base class instead.

Q247. Why is a transaction a class and not a struct? A class can be randomized with constraints, can carry methods (`copy`, `compare`, `print`), can be extended to create variants without editing the original, and is passed between components by handle without copying the data. A struct has none of these: no methods, no constraint blocks, no inheritance, and every assignment copies the whole value. Structs are for static bundles of signals in RTL.

Q248. Apart from resolving name clashes, what is `this` used for? To pass the current object to another object, for example `scb.register(this);` or `child = new(this);` so that a child component knows its parent; to return the object from a method so calls can be chained; and to make it explicit in long methods that a name is a property and not a local variable.

Q249. Can a class have no constructor? Can a constructor have arguments, a return type, or be virtual? If no `new` is written, the compiler provides an empty one that still initialises the properties and calls the base constructor. A constructor may have arguments, including default values. It has no return type (not even `void`); it implicitly returns the handle. It cannot be `virtual` or `static`, because it is called on the class, before any object exists.

Q250. Give a practical example of encapsulation with `local` and `protected`.

```
class alu_scoreboard;
  local     int unsigned fail_cnt;        // only this class can change the count
  protected alu_txn      exp_q [$];       // derived scoreboards may use the queue

  function int unsigned get_fail_cnt();   // read-only access for everyone else
    return fail_cnt;
  endfunction
endclass
```

No other component can reset the error count by accident, and the internal queue can be changed to an associative array later without affecting users of the class.

Q251. Give a practical example of abstraction.

```
virtual class base_driver;
  pure virtual task drive(alu_txn txn);   // every driver must implement this
  task run(mailbox #(alu_txn) mbx);       // common code written once
    alu_txn txn;
    forever begin
      mbx.get(txn);
      drive(txn);
    end
  endtask
endclass
```

The base class fixes the structure (get a transaction, drive it) and leaves the protocol-specific part to derived classes. `base_driver` cannot be constructed; a class that extends it and omits `drive` does not compile.

Q252. What is the difference between inheritance and composition, and how do you choose? Inheritance is an "is-a" relationship: `err_txn` is an `alu_txn`. Composition is a "has-a" relationship: `alu_env` has a driver and a monitor as member handles. Use inheritance when the new class can be used wherever the base class is expected; use composition to build larger things out of parts. An environment that extended the driver class would be a misuse of inheritance.

Q253. `randomize()` is called through a base-class handle that refers to a derived object. Which constraints apply? The constraints of the object's actual class: the base-class constraints plus those added or overridden in the derived class. `randomize()` is a built-in virtual method, so it acts on the real object. This is what makes the next question's pattern work.

Q254. How can a test change the kind of transaction the generator produces without editing the generator? The generator keeps a "blueprint" handle of the base type, randomizes it and sends a copy:

```
class alu_generator;
  alu_txn blueprint = new();
  task run();
    repeat (n) begin
      if (!blueprint.randomize()) $fatal(1, "randomize failed");
      gen2drv_mbx.put(blueprint.copy());
    end
  endtask
endclass

// in the test
class add_only_txn extends alu_txn;
  constraint add_c { op == ALU_ADD; }
endclass
...
add_only_txn t = new();
env.gen.blueprint = t;                    // base handle now refers to a derived object
```

Because `randomize()` and `copy()` are virtual, the generator now produces `add_only_txn` objects. The UVM factory automates exactly this substitution.

Q255. What are `const` properties in a class? A *global constant* is initialised in its declaration, `const int MAX = 16;`, and can never be changed; it is usually also `static`. An *instance constant* is declared without a value, `const int id;`, and must be assigned exactly once, in the constructor; after that it is read-only. An instance constant is useful for an ID that must not change during the life of the object.

Q256. Two classes each contain a handle of the other. How do you make this compile? With a forward type declaration before the first class: `typedef class alu_env;`. The compiler then accepts `alu_env` as a class name inside `alu_driver` before the full definition of `alu_env` appears later in the same scope.

Q257. What is the class scope resolution operator used for? `class_name::member` accesses things that belong to the class and not to an object: static properties and methods (`alu_txn::count`), and types, enums and parameters declared inside the class (`packet::kind_e`). It is also used to define an `extern` method outside the class (`function void alu_txn::print();`) and, with a package name, to pick a class out of a package (`alu_tb_pkg::alu_txn`).

Q258. What can a class not contain that a module can? A class cannot contain `initial` or `always` blocks, continuous assignments, module or interface instances, nets, or concurrent assertions. It can contain properties, tasks (which may consume time), functions, constraints, covergroups, immediate assertions and `fork`...`join`. Activity in a class starts only when some `initial` block in a module calls one of its tasks.

Q259. How are two objects compared? What does `h1 == h2` test, and can a method be called on a null handle? `h1 == h2` is true only if both handles refer to the same object (or both are `null`); it does not look at the contents. To compare contents, write a `compare()` method that compares each field (with `===` for four-state fields). A non-static method cannot be called on a null handle. A static method can, because it does not use the object: `alu_txn t; t.get_count();` works when `get_count` is static, though `alu_txn::get_count()` is the clearer form.

IPC-Based Questions

Q260. Why is inter-process communication needed in a class-based testbench? The generator, driver, monitor and scoreboard run as separate threads at the same time, each at its own pace: the generator works in zero time, the driver follows the clock, the monitor waits for DUT outputs. They must exchange data without losing or duplicating it and must not run ahead of each other. Mailboxes move the data and block when there is nothing to do, events signal milestones, and semaphores protect shared resources.

Q261. Why use a mailbox between the generator and the driver instead of a shared queue? A queue has no blocking: the driver would have to poll it (`wait (q.size() > 0)`), and two threads modifying a queue in the same time step can interfere. A mailbox's `get()` suspends the driver until an item is available and its `put()` can suspend the generator when a bounded mailbox is full, and the operations are atomic. A queue is the right choice inside one component (for example the scoreboard's list of expected results); a mailbox is the right choice between threads.

Q262. When would you use `peek()` instead of `get()`? When the item must stay in the mailbox until it has been fully processed. With a mailbox of size 1, the driver can `peek()` the transaction, drive it for several clocks, and only then `get()` it to remove it. During that time the generator's next `put()` stays blocked, so the generator is throttled to the driver's speed without any extra event.

Q263. What do `try_get()` and `try_put()` return? `try_put()` returns a positive value if the item was stored and 0 if the mailbox is full. `try_get()` and `try_peek()` return a positive value on success, 0 if the mailbox is empty, and a negative value if an item is present but its type does not match the argument (possible only with a non-parameterised mailbox). They never block, so they can be used inside functions.

Q264. Describe three ways to make the generator wait until the driver has finished a transaction. (1) An event: the driver triggers `drv_done` after driving and the generator waits for it after each `put()`. (2) A bounded mailbox of size 1 together with `peek()` in the driver, as in Q262. (3) A second mailbox in the opposite direction, through which the driver returns the transaction (or a response) and the generator does a blocking `get()` on it. UVM's `get_next_item()` and `item_done()` are a standardised form of this handshake.

Q265. What is `wait_order`? A statement that waits for several events to be triggered in a given order: `wait_order (req, gnt, done) $display("ok"); else $error("wrong order");`. It passes when all the events have occurred in that sequence and fails if one of them is triggered out of order. It is a compact protocol-order check.

Q266. How do you wait for any one of several events, and for all of them? Any one: `@(e1 or e2);` (also written `@(e1, e2)`), or a `fork` with one waiting branch per event followed by `join_any`. All of them, in any order and at different times: a `fork` ... `join` with one branch per event, each doing `wait (e.triggered)` or `@e`. `wait (e1.triggered && e2.triggered)` works only if both are triggered in the same time step, because `triggered` is cleared when time advances.

Q267. What is a race condition between testbench threads? Give an example and the fix. A race is when the result depends on the order in which the simulator runs threads that are ready at the same time. Example: one thread does `count = count + 1;` and another prints `count` in the same time step; the printed value depends on which runs first. Fixes: communicate through a mailbox instead of a shared variable, protect the shared variable with a semaphore, separate the threads in time with clocking blocks, or use `wait (e.triggered)` or `->>` for events.

Q268. What is a deadlock? Give an example with semaphores. A deadlock is when two or more threads each wait for something that only another of them can provide, so none can continue. Example: thread A takes semaphore `s1` and then asks for `s2`; thread B takes `s2` and then asks for `s1`. Each holds what the other needs. Prevention: always acquire resources in the same order, take both keys in one call where possible, and add a timeout around blocking calls so that a deadlock is reported instead of hanging silently.

Q269. A task starts a thread with `fork` ... `join_none` and returns. What happens to the thread? It keeps running. A thread started with `join_none` or left behind by `join_any` is not tied to the lifetime of the task that started it; it ends when its own statements finish, when it is killed by `disable fork`, `disable` or `process.kill()`, or when the simulation ends.

Q270. When does a simulation end if there are threads still waiting? Simulation ends when `$finish` is called or when no further events are scheduled. A thread blocked forever on a mailbox or an event does not keep the simulation alive by itself: if nothing else is scheduled (no clock, no pending delays), the simulation stops. With a free-running clock generator there is always a next event, so a testbench with a hung thread runs forever unless a watchdog calls `$finish`.

Q271. What is the danger of `disable task_name` or `disable block_name`? It stops every active execution of that named task or block, not only the one started by the caller. If two threads are both inside the same task and one of them executes `disable` on the task name, both are terminated. For killing specific threads use `disable fork` inside an isolating `fork` ... `join`, or a `process` handle and `kill()`.

Q272. Give an example where a semaphore with more than one key is needed. A DUT that can service at most two requests at a time, driven by four requesting threads: `semaphore slots = new(2);` and each thread

does `slots.get(1); ... slots.put(1);`, so at most two are active. A thread can also ask for several keys at once, `slots.get(2)`, to reserve the whole resource, and waits until both keys are free.

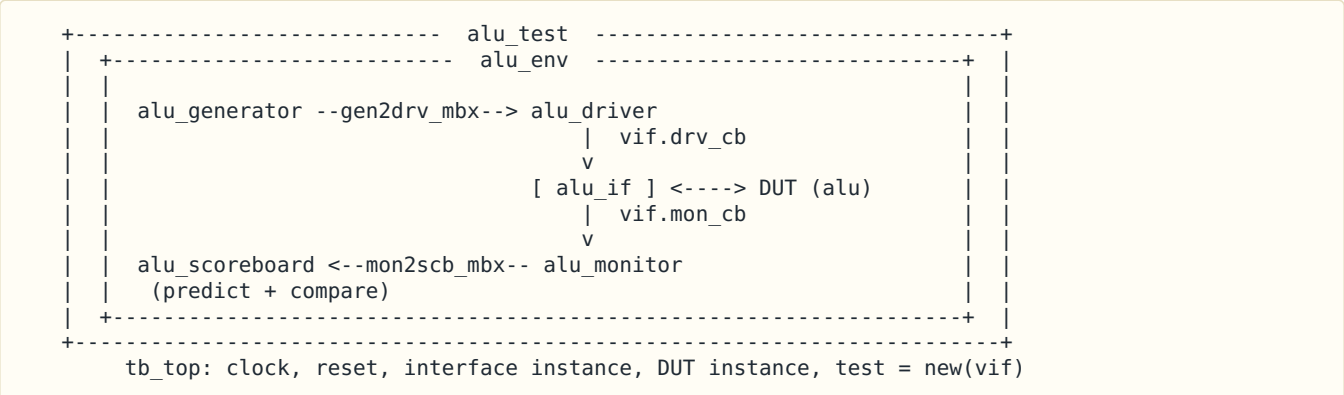
Q273. How is one event shared between two class objects? An event variable is a handle. Declare the event in the environment and pass it to both constructors, `gen = new(mbx, done); drv = new(mbx, done);`, each class storing it in its own `event` member; both members then refer to the same event object. Alternatively one object can assign the other's event directly: `drv.drv_done = gen.drv_done;`. If each class merely declares its own event and nothing connects them, they are two unrelated events.

Q274. At the end of a test the generator is finished. Why must the test not stop immediately, and how does it wait correctly? Transactions may still be waiting in `gen2drv_mbx`, in flight inside the DUT, or waiting in `mon2scb_mbx`. Stopping at once would skip their checks. The test should wait until the driver has driven everything (a count or `drv_done`), then until the scoreboard has compared the expected number of transactions (or its expected queue is empty), with a timeout in case the DUT never responds. A fixed "drain time" delay is a common but weaker substitute.

Q275. What is the difference between `#delay`, `@(event)` and `wait (condition)`? `#n` suspends the process for a fixed amount of simulation time. `@(x)` suspends until the next change (or the named edge) of `x`; it is edge-sensitive and always waits, even if `x` already has the desired value. `wait (cond)` is level-sensitive: it continues immediately if the condition is already true and otherwise suspends until it becomes true. `@(posedge ready)` misses the case where `ready` is already high; `wait (ready)` does not.

Layered Testbench Architecture Questions

Q276. Draw the layered testbench and describe the flow of one transaction.



The generator creates and randomizes an `alu_txn` and puts it in `gen2drv_mbx`. The driver gets it and, on the next clocking event, drives `a`, `b`, `op` and `valid_in` through the virtual interface. The DUT computes and one clock later presents `result` with `valid_out`. The monitor samples the inputs and the output from the interface, builds a new `alu_txn` and puts it in `mon2scb_mbx`. The scoreboard gets it, computes the expected result from the observed `a`, `b` and `op`, compares it with the observed `result` and records pass or fail.

Q277. Why is a layered testbench preferred to one flat testbench module? Each layer has one job and a defined connection to its neighbours, so it can be written, debugged, replaced and reused independently. A new test changes only the test layer; a protocol change touches only the driver and monitor; the checking is independent of how stimulus was created. A flat testbench mixes stimulus, timing and checking, so every change risks breaking everything and nothing can be reused in another project or at chip level.

Q278. Why do the components exchange transactions through mailboxes instead of calling each other's methods? Mailboxes decouple the components: the generator does not need a handle to the driver, does not depend on its method names, and the two can run at different speeds, the mailbox absorbing the difference. A component can be replaced, or a second consumer added, without touching the producer. UVM's TLM ports are a more formal version of the same decoupling.

Q279. Why does the scoreboard take its data from the monitor and not directly from the generator or driver? The monitor reports what actually happened on the DUT pins. If the driver has a bug (wrong timing, wrong bit order), the generator's copy of the transaction would not match what the DUT received, and the scoreboard would either report false errors or, worse, hide real ones. Taking both stimulus and response from the monitor also lets the same scoreboard work when the block is driven by other RTL at chip level, where no generator exists.

Q280. Where is the expected result computed? In a reference model (predictor), normally a function or class inside or next to the scoreboard. It takes the inputs observed by the monitor and computes what a correct DUT should output, written from the specification and independently of the RTL: for the ALU, a `case (op)` that computes a 9-bit result. Complex models are sometimes written in C and called through DPI. The model must not copy the RTL code, or it will contain the same bugs.

Q281. In what order does the environment build, connect and start its components? Build: construct the mailboxes and events, then construct each component. Connect: give the generator and driver the same `gen2drv_mbx` handle, the monitor and scoreboard the same `mon2scb_mbx` handle, and the driver and monitor the virtual interface. Run: start every component's `run()` task in parallel with `fork`. The shared objects are created in the environment because it is the only place that knows both ends of each connection.

Q282. How does the test know when to end? The generator triggers `gen_done` (or simply returns) after producing the required number of transactions. The environment then waits until the scoreboard has checked that number of transactions, protected by a timeout, and calls the scoreboard's report function and `$finish`. The forever-running driver, monitor and scoreboard threads are started with `join_none` or left behind by `join_any`, so they do not prevent the end. In UVM the same decision is made with objections.

```
task run();
  fork
    gen.run();  drv.run();  mon.run();  scb.run();
  join_any                                  // gen.run() is the only one that returns
  fork
    wait (scb.checked_cnt == gen.num_txn);
    #100000 $error("timeout: checked %0d of %0d", scb.checked_cnt, gen.num_txn);
  join_any
  disable fork;                             // also stops drv, mon and scb: intended here
  scb.report();
  $finish;
endtask
```

Q283. How do you add a new test without changing the environment? Write a new test class (or extend a base test) that changes only configuration: the number of transactions, constraint modes, an extended transaction placed in the generator's blueprint handle (Q254), or a different generator sequence. The environment, driver, monitor and scoreboard stay untouched. If a new test needs edits inside the driver, the testbench layering is wrong.

Q284. How should the components behave when reset is applied in the middle of a test? The driver stops driving, sets the interface to its idle values and discards or restarts the current transaction. The monitor abandons any partly collected transaction. The scoreboard flushes its queue of expected results, because the DUT has lost its state and those outputs will never appear. Each component watches `rst_n`, typically with a `fork` of the normal activity and a reset-wait, followed by `disable fork` and a restart after reset is released.

Q285. What is the difference between an active and a passive agent, and why does it matter for reuse? An agent groups the driver, monitor (and sequencer) for one interface. In active mode it drives and monitors; in passive mode it only monitors. At block level the agent on the DUT's inputs is active. At chip level those inputs are driven by another RTL block, so the same agent is switched to passive: the monitor, coverage and scoreboard are reused unchanged while the driver is disabled. This is the main reason the driver and monitor are kept strictly separate.

Q286. The DUT may return responses in a different order from the requests. How does the scoreboard handle it? Each transaction carries an identifier (a tag, or the address). The scoreboard stores expected transactions in an associative array indexed by that ID instead of a queue: `alu_txn exp [int unsigned];`. When a response arrives, it looks up the entry with the response's ID, compares, and deletes the entry. A response whose ID is not in the array is an error, and entries left at the end of the test are missing responses.

Q287. The DUT has a pipeline latency of several cycles and outputs stay in order. How does the scoreboard match inputs to outputs? With a queue used as a FIFO. When the monitor reports accepted inputs, the scoreboard computes the expected result and pushes it to the back of the queue. When the monitor reports an output, the scoreboard pops the front of the queue and compares. The latency does not need to be known; only the order matters. If the latency itself is a requirement, it is checked separately, by a timestamp in the transaction or by an assertion.

Q288. Where is functional coverage sampled, and why not in the generator? In the monitor, or in a coverage component fed by the monitor, when a complete transaction has been observed on the interface. Coverage must record what the DUT actually experienced. Sampling in the generator would count transactions that were never driven (for example dropped because of reset or a driver bug) and could not include response information such as the carry bit of the result.

Q289. What must a driver do besides copying transaction fields to signals? Follow the protocol timing: wait for reset to finish, drive only on the correct clock edge through the clocking block, assert `valid_in` for the right number of cycles, return the signals to idle values between transactions, wait for a ready or acknowledge signal if the protocol has back-pressure, and insert the delays requested by the transaction (idle cycles). It must also tell the generator when it is done if a handshake is used.

Q290. What belongs in `tb_top` and what does not? `tb_top` is the static part: clock and reset generation, the interface instance, the DUT instance connected to the interface, the construction of the test with the virtual interface, the call to `test.run()`, and waveform dumping. It does not contain stimulus, checking or anything specific to one test. Everything dynamic is in classes inside the package `alu_tb_pkg`.

Q291. A test fails in a layered testbench. How do you find which layer is at fault? Follow one transaction through the layers using its `id`. Print the transaction when the generator creates it, when the driver drives it, when the monitor observes it and when the scoreboard compares it. If generator and driver agree but the monitor shows different inputs, the fault is in the driver, the monitor or the interface connection; check the waveform at the interface. If the inputs agree everywhere and the result differs, compute the correct answer by hand: if the scoreboard's expectation is wrong the reference model is at fault, otherwise the DUT is.

Industry Scenario Questions

Scenario questions have no single correct answer. The interviewer is listening for a method: you reproduce the problem, narrow it down step by step, name the likely causes in order of probability, and say how you would confirm each one. Answer in numbered steps.

Debugging Scenarios

Q292. A test hangs: the simulation never finishes. How do you debug it?

1. Decide which kind of hang it is. If simulation time keeps increasing, some thread is waiting for something that never happens while the clock keeps running. If time is stuck and the CPU is busy, there is a zero-delay loop.
2. For a zero-delay loop: interrupt the simulator and look at where it is executing. Typical causes are a `forever` or `while` loop with no blocking statement on some path, `wait (e.triggered)` inside a loop, or combinational feedback in the RTL.
3. For a waiting hang: use the log to find the last message from each component (this is why components print with a time and a transaction `id`). The component that went silent first is the starting point.
4. Check the usual suspects: a `get()` on a mailbox nobody fills (two mailbox objects instead of one shared handle), an `@event` that missed its trigger, a `fork`...`join` in which one branch never ends, a semaphore key that was never returned, a driver waiting for a ready signal the DUT never asserts, reset never released, or a wrong end-of-test count.
5. In the waveform, check that the clock is running, reset is released, and whether the DUT is waiting for something from the testbench.
6. Add a watchdog timeout to the testbench so that future hangs end with an error message that says what was being waited for.

Q293. Functional coverage is stuck at 85% however many seeds are run. What do you do?

1. Open the coverage report and list the uncovered bins. Do not run more seeds first; if a bin has zero hits after hundreds of runs, more of the same will not help.
2. Classify each hole. (a) Unreachable: the bin describes something the design cannot do or the specification forbids; remove it with `ignore_bins` or `illegal_bins` after confirming with the specification. (b) Blocked by constraints: the stimulus cannot produce it; review the constraints (an over-tight range, a `dist` with weight 0, a conflicting constraint in a base class). (c) Reachable but improbable: for example a cross of two rare conditions, or a FIFO that never fills because reads are as frequent as writes.
3. For (b) and (c), write a targeted test: change the distribution weights, add a constraint in an extended transaction, or shape the timing (many writes with no reads to fill the FIFO).
4. If a few bins remain, write short directed tests for them.
5. Check that the coverage model itself is right: a wrong sampling event or a cross that includes meaningless combinations also shows up as permanent holes.

Q294. A test passes with one seed and fails with another. What does that tell you and how do you proceed? The failing seed generated a scenario the passing seed did not; that is constrained-random testing doing its job. 1. Rerun the failing seed to confirm that it fails every time; a random test must be reproducible with its seed. 2. Find the first error message and the transaction `id` involved, not the last one. 3. Compare that transaction with the specification: is the stimulus legal? If it is illegal, a constraint is missing (testbench bug). If it is

legal, compute the expected result by hand to decide whether the reference model or the DUT is wrong. 4. Look for what is special about it: a boundary value, back-to-back operations, an operation during or just after reset. 5. After the fix, rerun the same seed, keep it in the regression, and add a coverage bin for the scenario so that it is tracked from then on.

Q295. The test passes in RTL simulation but fails in gate-level simulation. Why might that be?

1. Uninitialised registers. RTL simulation can hide X through X-optimism (`if` and `case` take a definite branch); the netlist propagates X pessimistically. Trace the X back to the first flip-flop that is still X after reset: usually a register without reset that the design reads too early.
2. The testbench hides X in RTL: two-state variables in the monitor or `==` in the scoreboard.
3. Timing. With SDF delays annotated, the testbench may drive inputs too close to the clock edge (setup or hold violations make the flip-flop model output X), or sample outputs before they have settled. Drive and sample with proper skews.
4. Reset and clock sequencing that is too short for the real cells.
5. Differences between RTL and netlist: synthesis interpreted `full_case` / `parallel_case` pragmas or a missing sensitivity-list entry differently from simulation, or the testbench refers to internal signals that were renamed or optimised away.
6. Zero-delay gate simulation races in clock-gating or clock-divider logic.

Q296. Every scoreboard comparison fails, and the actual value always equals the expected value of the previous (or next) transaction. What is wrong? The data is right but the pairing is off by one. Causes, in order: (1) the monitor samples the output in the wrong cycle relative to the inputs, ignoring the DUT's one-cycle latency; (2) the monitor samples on the raw clock edge instead of the clocking block and catches the value from before or after the edge; (3) the scoreboard pushes an expected value for a cycle in which `valid_in` was not really accepted, or the DUT produced an extra output after reset, shifting the queue by one; (4) the first transaction was driven during reset and ignored by the DUT. Confirm by lining up one transaction in the waveform: the cycle the inputs were accepted, the cycle the output appeared, and the cycles in which the monitor captured each.

Q297. The nightly regression suddenly takes three times as long. How do you find out why?

1. Find out when it changed and what was checked in on that day (RTL, testbench, scripts, tool version, machine pool).
2. Compare a single test before and after: is the simulated time the same and the wall-clock time longer, or does the test now simulate longer? Longer simulated time means an end-of-test condition changed, for example tests now end on a timeout instead of on completion.
3. If the wall-clock time per simulated cycle went up, use the simulator's profiler. Typical causes: waveform dumping or full debug access switched on for all tests, verbose logging, an assertion or covergroup sampled on every clock on a wide signal, a scoreboard queue or associative array that is never emptied, a polling loop that wakes every time unit, or a `#1` clock where the period used to be larger.
4. Check the infrastructure: fewer machines, licences queuing, compilation no longer shared between tests.

Q298. `randomize()` fails in one test only. How do you debug it?

1. Read the solver's message: simulators print the set of conflicting constraints and the values of the state variables involved.
2. List everything that applies in that test: the class constraints, constraints inherited from base classes, constraints added by the extended transaction of this test, the inline `with` block, and any variable whose `rand_mode` is off (it now acts as a constant) or that is non-random by declaration.

3. Look for the contradiction: usually an inline constraint against a class constraint, or a state variable holding a value the constraints do not allow (for example a size left at 0).
4. Isolate by switching constraint blocks off one at a time with `constraint_mode(0)` until the call succeeds; the last one switched off is part of the conflict.
5. Fix by removing the contradiction, making the default constraint `soft`, or disabling it in this test. Check for width or sign problems in the expressions.
6. Make sure every `randomize()` call is checked, so that such a failure stops the test at the point of failure.

Q299. The DUT outputs are X at the start of every test. Is that a bug? Not necessarily. Before reset is applied, flip-flops are X, and that is expected. It is a bug if (1) outputs are still X after reset has been applied and released for the specified time: then a register feeding the output has no reset or reset is not connected; (2) an input of the DUT is undriven (Z) or X because the testbench did not initialise it, which is a testbench bug; (3) there is contention on a net. The testbench should hold all inputs at known idle values during reset, start checking only after reset, and have an assertion that control outputs are not unknown once reset is released.

Q300. A test fails in the regression but passes when you run it alone. What could be different? The seed (the regression uses a random seed; rerun with the seed from the regression log), the command-line arguments and plusargs, the compile options (coverage, assertions or optimisation switched on), the version of the code that was checked out, or files shared between tests (two tests writing to the same output or memory-image file). Reproduce by running the exact command line from the regression log, then remove differences one at a time.

Q301. Simulations of a long test run out of memory. What do you check in the testbench? Objects that are never released: a scoreboard queue or associative array from which matched entries are not deleted, a monitor that stores every transaction "for debug", a mailbox whose consumer is slower than its producer (an unbounded mailbox grows without limit), or threads created with `fork` ... `join_none` in a loop that never end. Also check waveform dumping of the whole design and large arrays that are declared as fixed-size where an associative array would store only the used entries.

Verification-Planning Scenarios

Q302. How would you verify a synchronous FIFO? Give the plan. *Features from the specification*: write when not full, read when not empty, data comes out in the order written, `full` and `empty` flags, behaviour on write-when-full and read-when-empty, simultaneous read and write, reset.

Stimulus: a transaction with random `wr_en`, `rd_en`, `din` and idle cycles; constraints or weights to bias towards filling (more writes), draining (more reads) and balanced traffic.

Checking: a scoreboard with a queue as reference model: push `din` on every accepted write, pop and compare with `dout` on every accepted read (with `!==`). Assertions: never `full` and `empty` together; `empty` after reset; no change of the pointer on write-when-full or read-when-empty; `full` asserted exactly when the count equals the depth.

Coverage: FIFO empty, full, and levels in between; full→not full and empty→not empty transitions; write when full; read when empty; simultaneous read and write at empty, at full and in between; the pointers wrapping around; data values 0x00 and 0xFF.

Tests: reset test, fill then drain, alternating write and read, random traffic with several weightings, overflow and underflow attempts, reset in the middle of traffic.

Exit criteria: all tests pass, 100% functional coverage, code coverage reviewed with justified exclusions.

Q303. How would you verify a round-robin arbiter with four requesters? *Checks*: at most one grant at a time (`$onehot0(gnt)`), a grant only to a requester that is requesting, a grant within one cycle (or the specified latency) whenever any request is present, and fairness: after requester *i* is served, the next grant goes to the next requesting index in circular order, and no requester waits for more than three other grants. *Reference model*: a small function that keeps the index of the last grant and computes the next expected one. *Stimulus*: random request vectors, with directed patterns for all requesting at once, a single requester, a requester that drops its request, and requests arriving in the same cycle. *Coverage*: each requester granted; each ordered pair "grant to i followed by grant to j"; all sixteen request patterns; reset during a grant.

Q304. How would you verify a 4-bit up/down counter with load and enable? *Features*: reset value, count up, count down, hold when not enabled, load a value, priority between load and count, wrap-around from 15 to 0 and from 0 to 15. *Checking*: a reference model in the scoreboard with one `case` on the control inputs evaluated each clock, compared with the DUT output; assertions such as "when enabled, counting up and not loading, the next value is the current value plus one". *Coverage*: every count value; the transitions 15→0 and 0→15; load of the boundary values 0 and 15; load while counting; each control combination; reset at a non-zero count. *Stimulus*: random control signals with the enable weighted high so that the counter actually moves, plus a directed test that counts through the full range in both directions.

Q305. All tests pass, but you suspect the testbench is not really checking anything. How do you gain confidence?

1. Inject a bug on purpose: change one operation in the DUT (or force a signal) and confirm that a test fails. If nothing fails, the checker is not working.
2. Look at the scoreboard counters: the number of comparisons must equal the number of transactions sent. "0 errors out of 0 comparisons" is a classic false pass.
3. Check that the comparison uses `!==`, that the monitor uses four-state types, and that the scoreboard reports an error for leftover expected items.
4. Check assertion coverage: assertions whose antecedent never fired have checked nothing.
5. Review the reference model against the specification, not against the RTL.

Q306. Functional coverage and code coverage are both 100%, yet a bug is found later. How is that possible? Coverage shows that scenarios were exercised, not that the results were checked. Possible reasons: the scoreboard did not check the affected output or field; the feature was missing from the verification plan, so no coverpoint existed for it (functional coverage is only as complete as the plan); the bug needs a combination or sequence that was not crossed; the reference model had the same misunderstanding of the specification as the RTL; or the bug is outside functional simulation (timing, clock-domain crossing, power sequencing). The lesson is that coverage must be paired with checking and with reviews of the plan.

Q307. You find a mismatch that you believe is an RTL bug. What do you do before and after reporting it? Before: make sure it is not a testbench problem by checking the stimulus against the specification and computing the expected value by hand; reduce the failure to the smallest test and a fixed seed; identify the first wrong signal in the waveform. Report with the test name, seed, command line, simulation time of the failure, expected and actual values, the specification section, and the waveform location. After the designer's fix: rerun the failing seed, run the full regression to make sure nothing else broke, keep the test and seed in the regression, and add a coverage bin or assertion for the scenario.

Q308. The schedule allows two more days and verification is not complete. How do you prioritise? By risk. First make sure every feature has at least a basic passing test and that the regression is clean. Then spend the time where a bug would be most damaging or most likely: the main data path, new or recently changed logic, complex control such as corner cases of flags and counters, and anything that failed recently. Close the largest

coverage holes with targeted tests. List what remains unverified, with the risk of each item, and tell the lead clearly. Do not report a block as done by quietly excluding the unfinished parts.

Frequently Asked Fresher Questions

Fresher interviews mix language questions with questions about fundamentals and about how you work. Short, honest, structured answers do best.

Q309. Explain your project. Prepare a two-minute answer with this structure. (1) What the DUT does, in two sentences: "an 8-bit ALU with eight operations; inputs are registered with a valid signal and the 9-bit result appears one clock later". (2) The testbench architecture: generator, driver, monitor, scoreboard, environment, connected with mailboxes and a virtual interface; draw it if a board is available. (3) Your own contribution: which parts you wrote. (4) How it checks: reference model in the scoreboard, assertions, functional coverage and the coverage you reached. (5) One bug you found and how you debugged it; this is the part interviewers remember. Be ready for follow-ups on anything you mention: if you say "mailbox", expect "why not a queue?".

Q310. What is the difference between verification, validation and testing? Verification is done before manufacturing, on the design description (RTL or gates), by simulation, formal methods and emulation: "did we build the design correctly, according to the specification?". Validation is done on the real silicon or a prototype, in the real system with real software: "does the product do what the customer needs?". Testing (manufacturing test) checks every manufactured chip for fabrication defects using scan patterns and built-in self-test; it assumes the design is correct.

Q311. What is the difference between simulation and synthesis? Simulation executes the RTL and the testbench in software to observe behaviour over time; it supports the whole language, including delays, classes and system tasks. Synthesis translates the synthesizable subset of the RTL into a netlist of gates from a technology library, under timing, area and power constraints. Code that simulates is not necessarily synthesizable, and synthesizable code can simulate differently from the gates if it is written badly (incomplete sensitivity lists, mixed blocking and nonblocking assignments), which is called a simulation-synthesis mismatch.

Q312. Why do you want to work in verification rather than design? Give a truthful answer in your own words. Points that are true of the job and worth mentioning: verification takes the majority of the effort on a chip project; it requires understanding the whole design and its specification, not one block; it combines hardware knowledge with software skills (OOP, randomization, debugging); and finding a bug before tape-out has a direct, visible value. Avoid saying that verification is easier or that it is a way into design.

Q313. What is your approach to debugging a failing test?

1. Reproduce it with the same seed and options. 2. Read the log from the top and find the first error, not the last. 3. Decide whether the stimulus was legal. 4. Follow the failing transaction through the testbench layers by its ID to find where expected and actual first differ. 5. Open the waveform at that time and trace the wrong signal backwards to its cause. 6. Form one hypothesis and test it; do not change several things at once. 7. Decide whether it is a testbench, reference-model, specification or RTL problem and fix or report it. 8. Rerun the seed and then the regression.

Q314. What is a self-checking testbench? A testbench that decides by itself whether the DUT's outputs are correct, using a scoreboard with a reference model and assertions, and prints a clear PASS or FAIL. Nobody has to inspect waveforms. This is essential for regressions of thousands of random tests.

Q315. What is a verification plan and what does it contain? The document, written from the specification before the testbench, that defines what will be verified and how. It contains the list of features to verify, the testbench architecture, the stimulus strategy (random and directed tests), the checkers and assertions for each feature, the functional coverage model that proves each feature was exercised, the list of tests, the exit criteria (coverage targets, no open bugs) and the schedule. It is the reference for answering "are we done?".

Q316. What is a regression? The complete set of tests for a design, run automatically (usually nightly) with many random seeds, whenever the RTL or the testbench changes. Its purposes are to detect that a change broke something that used to work, and to accumulate coverage. Results are tracked as pass rate and coverage trends.

Q317. What is a corner case? Give examples for a FIFO. A legal but unusual situation at the boundary of the design's behaviour, where bugs are more likely because several conditions meet. For a FIFO: a write when it is full; a read when it is empty; a simultaneous read and write when exactly full or exactly empty; the pointers wrapping from the last location to the first; a single entry written and read back immediately; reset while it is not empty.

Q318. Give one example each of a compile-time, an elaboration-time and a run-time error in a class-based testbench. Compile time: assigning a base-class handle to a derived-class handle without `$cast`, or calling a method that the class does not have. Elaboration time: a module instance whose port list does not match the module, or an interface passed to a port of a different interface type. Run time: using a null handle, a failed `$cast` called as a task, a `randomize()` failure, an out-of-range array index, or an assertion failure.

Q319. When do you debug with the log and when with the waveform? The log is the first tool for a class-based testbench: transactions, comparisons and errors are printed with times and IDs, and classes do not appear in waveforms the way signals do. The waveform is used once the log has pointed to a time and a signal: to see exact cycle timing on the interface, to trace a wrong DUT output back through the RTL, and for X propagation. Start with the log to find *when*, then the waveform to find *why*.

Q320. What is the difference between `$finish` and `$stop`, and between `$fatal`, `$error`, `$warning` and `$info`? `$finish` ends the simulation. `$stop` pauses it and returns control to the interactive simulator prompt, from which it can be continued. The severity tasks all print a message with the time and scope: `$info` is informational, `$warning` flags something suspicious, `$error` reports a failure and lets the simulation continue (the simulator counts errors), and `$fatal` reports a failure and ends the simulation immediately. A scoreboard mismatch is an `$error`; a null virtual interface or a failed randomization is a `$fatal`.

Q321. What does `` `timescale 1ns/1ps `` mean? The first value is the time unit: a delay written as `#1` means 1 ns. The second is the time precision: delays are rounded to the nearest 1 ps, so `#1.2345` becomes 1.235 ns. The precision cannot be larger than the unit. Inside a module, `timeunit 1ns; timeprecision 1ps;` is the SystemVerilog way to say the same without depending on the order in which files are compiled.

Q322. What are setup time and hold time? Does functional simulation check them? Setup time is how long the data input of a flip-flop must be stable before the active clock edge; hold time is how long it must stay stable after the edge. A violation can make the flip-flop metastable. RTL functional simulation has no gate delays and does not check them. They are verified by static timing analysis, and by gate-level simulation with back-annotated SDF delays, where the timing checks in the cell models report violations.

Q323. What is the difference between a Moore and a Mealy state machine? In a Moore machine the outputs depend only on the current state, so they change only after a clock edge and are glitch-free with respect to inputs. In a Mealy machine the outputs depend on the current state and the current inputs, so they can respond in the same cycle as the input, usually with fewer states, but a change on an input passes straight to the output. In the sequence detector of Q209, `det = (state == S101) && din` is a Mealy output.

Q324. What is the difference between synchronous and asynchronous reset, and how do you verify reset? A synchronous reset takes effect only at a clock edge: `always_ff @(posedge clk) if (!rst_n) ...`. An asynchronous reset takes effect as soon as it is asserted, without a clock: `always_ff @(posedge clk or negedge rst_n)`; its release must still be synchronised to the clock. To verify reset: check that every output and important register has its specified value after reset, with no X; apply reset in the middle of activity, not only at time 0, and

check that the DUT recovers and works afterwards; for asynchronous reset, assert it away from the clock edge and check that the outputs respond immediately.

Q325. How does a synchronous FIFO generate `full` and `empty`? With a write pointer, a read pointer and a way to tell full from empty when the pointers are equal. One method keeps a counter of the number of entries: increment on write, decrement on read, unchanged on both; `empty` is count == 0 and `full` is count == DEPTH. The other makes the pointers one bit wider than the address: `empty` when the pointers are identical, `full` when they differ only in the extra top bit. A write when full and a read when empty must not change the pointers.

Q326. How much UVM should a fresher know? Know what UVM is and why it exists (Q55), the main components and how they correspond to the plain SystemVerilog layered testbench (sequence item = transaction, sequence = generator, sequencer and driver, monitor, agent, scoreboard, environment, test), the idea of phases, the factory and the configuration database, and how the driver gets items (`get_next_item`, `item_done`). Be honest about the depth: "I have built the layered testbench in plain SystemVerilog and understand how each part maps to UVM" is a strong fresher answer; claiming UVM experience you do not have is quickly exposed.

Last-Day Revision Checklist

Read each line and answer it aloud in two sentences. If you cannot, go back to the question or module in brackets.

Language basics

- Two-state and four-state types; default values; `logic` vs `wire` (Q3, M1).
- `==` vs `===` vs `==?`; which one in a scoreboard (M1, Q177).
- Blocking vs nonblocking assignment (Q56, Q168).
- Fixed, dynamic, associative arrays and queues; their methods (Q57, Q9, Q61).
- Task vs function; `automatic`; `ref` (Q13, Q15, Q65).

OOP

- Class, object, handle, `null`, `new` (Q18 to Q20).
- Shallow vs deep copy (Q69, Q156).
- Inheritance, `super`, constructor order (Q71, Q158).
- Virtual methods and polymorphism; `$cast` (Q72, Q73, Q157).
- Abstract class, pure virtual; static members; `local` / `protected` (Q28, Q23, Q24).

Randomization

- `rand` vs `randc`; checking `randomize()` (Q29, Q30).
- `inside`, `dist`, implication, `solve before`, `soft`, `unique` (Q78 to Q82).
- `constraint_mode` vs `rand_mode`; `pre_randomize` / `post_randomize` (Q35, Q36, Q170).
- Array constraints with `foreach`, `size()`, `sum()` (Q84, Q236 to Q243).
- Width, sign and chained-comparison traps in constraints (Q182, Q233).

IPC

- `join`, `join_any`, `join_none`; `disable fork`; `wait fork` (Q44, Q89, Q162).
- The loop-variable trap with `join_none` (Q92, Q163).

- Mailbox methods; bounded vs unbounded (Q41, Q91, Q165).
- Event: `->`, `->>`, `@`, `wait(triggered)` (Q88, Q90, Q166).
- Semaphore; deadlock; race condition (Q42, Q267, Q268).

Testbench

- Interface, modport, clocking block, virtual interface (Q45 to Q48, Q94).
- Why the clocking block removes races; event regions (Q111, Q113).
- Role of each component and the flow of one transaction (Q49, Q276).
- Why the scoreboard listens to the monitor; how the test ends (Q279, Q282).
- One object per transaction in generator and monitor (Q172, Q183).

Coverage and assertions

- Code vs functional coverage; bins, cross, `ignore_bins`, `illegal_bins` (Q39, Q99, Q101).
- Immediate vs concurrent; `|->` vs `|=>`; `disable iff`; `$past`, `$rose` (Q50, Q51, Q104, Q105).
- Vacuous success (Q142).

Yourself

- Your project in two minutes, with one bug you found (Q309).
- Your debugging method in numbered steps (Q313).
- Verification plan for a FIFO (Q302).

Coding Exercises with Solutions

These ten exercises are larger than the write-the-code questions. Each has a complete, self-contained solution that is also saved as a `.sv` file in the `examples/module-12/` folder. Every program prints a result that does not depend on the random seed, so you can check your simulator's output against the expected output shown.

Attempt each exercise from the problem statement alone before reading the solution. In an interview you would be given 20 to 40 minutes for one of these.

Exercise 1: A Parameterised FIFO Class

Problem. Write a class that models a FIFO of any element type and any depth. `push` must refuse an item when the FIFO is full and `pop` must report when it is empty. Test it with a depth of 3 by pushing four bytes and popping until empty.

fx01_fifo_class.sv

```
module fx01_fifo_class;

  class fifo #(type T = int, int DEPTH = 4);
    local T items [$];

    function bit is_full();
      return items.size() == DEPTH;
    endfunction

    function bit is_empty();
      return items.size() == 0;
    endfunction

    function int count();
      return items.size();
    endfunction

    function bit push(T item);
      if (is_full()) return 0;
      items.push_back(item);
      return 1;
    endfunction

    function bit pop(output T item);
      if (is_empty()) return 0;
      item = items.pop_front();
      return 1;
    endfunction
  endclass

  fifo #(bit [7:0], 3) f = new();
  bit [7:0] d;
  bit       ok;

  initial begin
    for (int i = 1; i <= 4; i++) begin
      d  = 8'(i * 16);
      ok = f.push(d);
      $display("push %h -> ok=%b count=%0d full=%b", d, ok, f.count(), f.is_full());
    end
    while (f.pop(d))
      $display("pop  %h    count=%0d empty=%b", d, f.count(), f.is_empty());
  end

endmodule
```

EXPECTED OUTPUT

```
push 10 -> ok=1 count=1 full=0
push 20 -> ok=1 count=2 full=0
push 30 -> ok=1 count=3 full=1
push 40 -> ok=0 count=3 full=1
pop  10    count=2 empty=0
pop  20    count=1 empty=0
pop  30    count=0 empty=1
```

Points to explain. The queue is `local`, so users can reach it only through the methods (encapsulation). Items enter at the back and leave at the front (first in, first out). `fifo #(bit [7:0], 3)` is one specialisation of the generic class. The fourth push is refused and the value `40` never appears on the pop side.

Exercise 2: A Synchronous FIFO and a Self-Checking Testbench

Problem. Write the RTL of `sync_fifo` (8 bits wide, 16 deep, with `full` and `empty`), and a directed self-checking testbench that uses a queue as the reference model. The testbench must fill the FIFO, attempt an overflow, drain it, attempt an underflow, and report the number of checks and errors.

fx02_sync_fifo_tb.sv

```
module sync_fifo #(
  parameter int WIDTH = 8,
  parameter int DEPTH = 16
) (
  input  logic             clk,
  input  logic             rst_n,
  input  logic             wr_en,
  input  logic             rd_en,
  input  logic [WIDTH-1:0] din,
  output logic [WIDTH-1:0] dout,
  output logic             full,
  output logic             empty
);
  localparam int AW = $clog2(DEPTH);

  logic [WIDTH-1:0] mem [DEPTH];
  logic [AW-1:0]    wr_ptr, rd_ptr;
  logic [AW:0]      count;
  logic             do_wr, do_rd;

  assign full  = (count == DEPTH);
  assign empty = (count == 0);
  assign do_wr = wr_en && !full;      // a write when full is ignored
  assign do_rd = rd_en && !empty;     // a read when empty is ignored

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      wr_ptr <= '0;
      rd_ptr <= '0;
      count  <= '0;
      dout   <= '0;
    end
    else begin
      if (do_wr) begin
        mem[wr_ptr] <= din;
        wr_ptr      <= wr_ptr + 1'b1;
      end
      if (do_rd) begin
        dout   <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1'b1;
      end
      case ({do_wr, do_rd})
        2'b10  : count <= count + 1'b1;
        2'b01  : count <= count - 1'b1;
```

```
        default: ;                    // both or neither: count unchanged
      endcase
    end
  end
endmodule

module fx02_sync_fifo_tb;

  logic       clk = 0;
  logic       rst_n, wr_en, rd_en;
  logic [7:0] din, dout;
  logic       full, empty;

  logic [7:0] model [$];              // reference model
  int         checks, errors;

  sync_fifo dut (.*);

  always #5 clk = ~clk;

  // Stimulus changes on the falling edge, so it is stable at the rising edge.
  task automatic write(input logic [7:0] data);
    @(negedge clk);
    wr_en = 1;
    din   = data;
    if (!full) model.push_back(data); // the DUT will accept it only if not full
    @(negedge clk);
    wr_en = 0;
  endtask

  task automatic read();
    logic [7:0] expected;
    bit         accepted;
    @(negedge clk);
    rd_en    = 1;
    accepted = !empty;
    if (accepted) expected = model.pop_front();
    @(negedge clk);                   // dout was updated at the rising edge in between
    rd_en = 0;
    if (accepted) begin
      checks++;
      if (dout !== expected) begin
        errors++;
        $display("[%0t] FAIL: dout=%h expected=%h", $time, dout, expected);
      end
    end
  endtask

  initial begin
    rst_n = 0;  wr_en = 0;  rd_en = 0;  din = '0;
    repeat (2) @(negedge clk);
    rst_n = 1;
    $display("after reset    : full=%b empty=%b", full, empty);

    for (int i = 0; i < 16; i++) write(8'hA0 + i);
    $display("after 16 writes: full=%b empty=%b", full, empty);

    write(8'hFF);                     // overflow attempt: must be ignored
    repeat (16) read();
    $display("after 16 reads : full=%b empty=%b", full, empty);

    read();                           // underflow attempt: must be ignored
    write(8'h11);
    read();

    $display("checks=%0d errors=%0d model_left=%0d", checks, errors, model.size());
    if (errors == 0 && checks == 17 && model.size() == 0) $display("TEST PASSED");
    else                                                  $display("TEST FAILED");
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
after reset    : full=0 empty=1
after 16 writes: full=1 empty=0
after 16 reads : full=0 empty=1
checks=17 errors=0 model_left=0
TEST PASSED
```

Points to explain.

- The pointer for a 16-deep FIFO is 4 bits, so it wraps naturally. The counter is one bit wider so that it can hold the value 16.
- `dout` is registered: the data appears after the clock edge at which the read was accepted. The testbench therefore checks `dout` one half-cycle after that edge.
- The reference model mirrors the DUT's acceptance rule. On the overflow attempt the model does not push `FF`; if the DUT wrongly stored it, a later read would mismatch.
- The final check is more than "errors == 0": it also requires the expected number of checks and an empty model, so a testbench that compared nothing cannot pass.

Exercise 3: Transaction Copy and Compare

Problem. Write the `alu_txn` class with `print`, `copy` and `compare`. Show the difference between assigning a handle and copying an object.

fx03_txn_copy_compare.sv

```
package fx03_pkg;

  typedef enum logic [2:0] {
    ALU_ADD, ALU_SUB, ALU_AND, ALU_OR, ALU_XOR, ALU_NOT, ALU_SHL, ALU_SHR
  } alu_op_e;

  class alu_txn;
    rand bit [7:0] a, b;
    rand alu_op_e  op;
    logic [8:0]    result;
    int unsigned   id;

    virtual function void print(string tag);
      $display("%s: id=%0d a=%0d b=%0d op=%s result=%0d", tag, id, a, b, op.name(), result);
    endfunction

    virtual function alu_txn copy();
      alu_txn c = new();
      c.a      = a;
      c.b      = b;
      c.op     = op;
      c.result = result;
      c.id     = id;
      return c;
    endfunction

    virtual function bit compare(alu_txn rhs);
      if (rhs == null) return 0;
      return (a == rhs.a) && (b == rhs.b) && (op === rhs.op) && (result === rhs.result);
    endfunction
  endclass

endpackage

module fx03_txn_copy_compare;
  import fx03_pkg::*;
```

```
  alu_txn t1, t2, t3;

  initial begin
    t1 = new();
    t1.id = 1;  t1.a = 200;  t1.b = 100;  t1.op = ALU_ADD;  t1.result = 300;

    t2 = t1;                 // second handle to the same object
    t3 = t1.copy();          // second object

    t2.a = 5;                // also changes what t1 sees
    t3.b = 7;                // changes only the copy

    t1.print("t1");
    t2.print("t2");
    t3.print("t3");
    $display("t1 == t2 (handles): %b   t1 == t3 (handles): %b", t1 == t2, t1 == t3);
    $display("t1.compare(t2): %b   t1.compare(t3): %b", t1.compare(t2), t1.compare(t3));
  end
endmodule
```

EXPECTED OUTPUT

```
t1: id=1 a=5 b=100 op=ALU_ADD result=300
t2: id=1 a=5 b=100 op=ALU_ADD result=300
t3: id=1 a=200 b=7 op=ALU_ADD result=300
t1 == t2 (handles): 1   t1 == t3 (handles): 0
t1.compare(t2): 1   t1.compare(t3): 0
```

Points to explain. `==` on handles tests identity; `compare()` tests contents. The four-state fields are compared with `===` so that an X in `result` is a mismatch instead of an unknown. The methods are virtual so that an extended transaction can override them and still be handled through an `alu_txn` handle.

Exercise 4: Polymorphism with an Abstract Base Class

Problem. Model ALU operations as classes. An abstract base class declares a pure virtual `calc` function; one derived class per operation implements it. Store the objects in a queue of base handles and call them in a loop.

`fx04_polymorphism.sv`

```
module fx04_polymorphism;

  virtual class alu_operation;
    string name;

    function new(string name);
      this.name = name;
    endfunction

    pure virtual function logic [8:0] calc(logic [7:0] a, logic [7:0] b);

    function void show(logic [7:0] a, logic [7:0] b);
      $display("%-4s %0d, %0d -> %0d", name, a, b, calc(a, b));
    endfunction
  endclass

  class op_add extends alu_operation;
    function new();
      super.new("add");
    endfunction
    virtual function logic [8:0] calc(logic [7:0] a, logic [7:0] b);
      logic [8:0] r;
      r = a + b;
      return r;
    endfunction
  endclass
```

```
  class op_sub extends alu_operation;
    function new();
      super.new("sub");
    endfunction
    virtual function logic [8:0] calc(logic [7:0] a, logic [7:0] b);
      logic [8:0] r;
      r = a - b;
      return r;
    endfunction
  endclass

  class op_xor extends alu_operation;
    function new();
      super.new("xor");
    endfunction
    virtual function logic [8:0] calc(logic [7:0] a, logic [7:0] b);
      return {1'b0, a ^ b};
    endfunction
  endclass

  alu_operation ops [$];
  op_add        h_add;
  op_sub        h_sub;
  op_xor        h_xor;

  initial begin
    h_add = new();
    h_sub = new();
    h_xor = new();
    ops.push_back(h_add);          // derived handles stored as base handles
    ops.push_back(h_sub);
    ops.push_back(h_xor);

    foreach (ops[i]) ops[i].show(200, 100);
    ops[1].show(100, 200);
  end

endmodule
```

EXPECTED OUTPUT

```
add  200, 100 -> 300
sub  200, 100 -> 100
xor  200, 100 -> 172
sub  100, 200 -> 412
```

Points to explain. `show` is written once in the base class and calls `calc`; because `calc` is virtual, each object runs its own version. `alu_operation` cannot be constructed. Adding a new operation means adding one class, with no change to the loop. The results are 9 bits wide: 100 - 200 is -100, which as a 9-bit pattern is 512 - 100 = 412 (bit 8 is the borrow). 200 XOR 100 is `1100_1000 ^ 0110_0100` = `1010_1100` = 172.

Exercise 5: Constraints with a Procedural Self-Check

Problem. Write a class with a random dynamic array of 4 to 8 elements such that every element is a multiple of 5 between 10 and 200, the elements are strictly ascending, and their sum is at most 600. Verify the constraints by checking 50 randomizations with ordinary procedural code. Then show a call that cannot be solved.

fx05_constraints_check.sv

```
module fx05_constraints_check;

  class sorted_arr;
    rand bit [7:0] a [];

    constraint size_c { a.size() inside {[4:8]}; }
    constraint elem_c { foreach (a[i]) { a[i] inside {[10:200]}; a[i] % 5 == 0; } }
    constraint asc_c  { foreach (a[i]) if (i > 0) a[i] > a[i-1]; }
    constraint sum_c  { a.sum() with (int'(item)) <= 600; }

    // The same rules written procedurally, as an independent check
    function bit is_legal();
      int total;
      total = 0;
      if (a.size() < 4 || a.size() > 8) return 0;
      foreach (a[i]) begin
        if (a[i] < 10 || a[i] > 200 || a[i] % 5 != 0) return 0;
        if (i > 0 && a[i] <= a[i-1])                  return 0;
        total += a[i];
      end
      return total <= 600;
    endfunction
  endclass

  sorted_arr s;
  int        solver_failures, illegal_results, ok;

  initial begin
    s = new();
    repeat (50) begin
      if (!s.randomize())      solver_failures++;
      else if (!s.is_legal())  illegal_results++;
    end
    $display("50 calls: solver failures=%0d illegal results=%0d",
             solver_failures, illegal_results);

    // Eight ascending multiples of 5 starting at 180 or more do not exist below 200
    ok = s.randomize() with { a.size() == 8; a[0] >= 180; };
    $display("over-constrained call returned %0d", ok);

    if (solver_failures == 0 && illegal_results == 0 && ok == 0) $display("TEST PASSED");
    else                                                         $display("TEST FAILED");
  end

endmodule
```

EXPECTED OUTPUT

```
50 calls: solver failures=0 illegal results=0
over-constrained call returned 0
TEST PASSED
```

The simulator also prints a constraint-solver warning for the last call; its wording differs between tools.

Points to explain. The size is solved first, then the elements. `sum()` is widened with `int'(item)` because eight 8-bit values can exceed 255. Ascending order already implies that all elements differ. The inline constraints leave only 180, 185, 190, 195 and 200 as candidates, five values for eight strictly ascending positions, so the solver reports failure and returns 0. Writing the rules a second time procedurally is how constraint classes are unit-tested in practice.

Exercise 6: A Three-Stage Pipeline with Mailboxes

Problem. A producer sends the numbers 1, 2 and 3, one every 10 time units. A processing stage doubles each number, taking 3 time units per item. A consumer prints what it receives. Connect them with two mailboxes and end the test when the consumer has received three items.

fx06_mailbox_pipeline.sv

```
module fx06_mailbox_pipeline;

  mailbox #(int) stage1_mbx = new();
  mailbox #(int) stage2_mbx = new();

  task automatic producer(int n);
    for (int i = 1; i <= n; i++) begin
      #10;
      stage1_mbx.put(i);
      $display("[%0t] producer : sent %0d", $time, i);
    end
  endtask

  task automatic doubler();
    int v;
    forever begin
      stage1_mbx.get(v);        // blocks while there is no work
      #3;
      stage2_mbx.put(2 * v);
    end
  endtask

  task automatic consumer(int n);
    int v;
    repeat (n) begin
      stage2_mbx.get(v);
      $display("[%0t] consumer : got %0d", $time, v);
    end
  endtask

  initial begin
    fork
      producer(3);
      doubler();
    join_none
    consumer(3);                // returns after the third item
    $display("[%0t] done", $time);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[10] producer : sent 1
[13] consumer : got 2
[20] producer : sent 2
[23] consumer : got 4
[30] producer : sent 3
[33] consumer : got 6
[33] done
```

Points to explain. No stage knows about the others; each only knows its mailboxes. `get()` blocks, so there is no polling. The `doubler` task never ends, so it is started with `join_none`, and the test ends on the consumer's count, not on the producer finishing (at time 30 the last item is still inside the pipeline).

Exercise 7: Three Users, Two Keys

Problem. Three threads need a resource that can serve two users at a time. Each uses it for 10 time units. The threads request the resource at times 0, 1 and 2. Use a semaphore and print when each thread enters and leaves.

fx07_semaphore.sv

```
module fx07_semaphore;

  semaphore slots = new(2);

  task automatic user(string name, int start);
    #start;
    slots.get(1);
    $display("[%0t] %s enters", $time, name);
    #10;
    $display("[%0t] %s leaves", $time, name);
    slots.put(1);
  endtask

  initial begin
    fork
      user("A", 0);
      user("B", 1);
      user("C", 2);
    join
    $display("[%0t] all done", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] A enters
[1] B enters
[10] A leaves
[10] C enters
[11] B leaves
[20] C leaves
[20] all done
```

Points to explain. A and B take the two keys. C asks at time 2 and blocks until A returns a key at time 10. At no time are more than two users inside. If `user` forgot `put(1)`, C would block forever and `join` would never complete.

Exercise 8: Killing Slow Threads with Process Handles

Problem. Start three jobs in parallel that take 10, 40 and 90 time units. At time 50, report which jobs have finished and kill those that have not.

fx08_process_control.sv

```
module fx08_process_control;

  string  names [3] = '{"fast", "medium", "slow"};
  int     durs  [3] = '{10, 40, 90};
  process p [3];

  task automatic job(string name, int dur);
    #dur;
    $display("[%0t] %s finished", $time, name);
  endtask

  initial begin
    foreach (p[i])
      fork
        automatic int k = i;
        begin
          p[k] = process::self();     // remember the handle of this thread
          job(names[k], durs[k]);
        end
      join_none

    #50;                              // the deadline
    foreach (p[i])
      if (p[i].status() == process::FINISHED)
        $display("[%0t] %s: finished in time", $time, names[i]);
      else begin
        $display("[%0t] %s: still running, killed", $time, names[i]);
        p[i].kill();
      end

    #100 $display("[%0t] end of test", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[10] fast finished
[40] medium finished
[50] fast: finished in time
[50] medium: finished in time
[50] slow: still running, killed
[150] end of test
```

Points to explain. Each thread stores its own `process` handle as its first action. `status()` tells the parent the state of each thread, and `kill()` terminates only the selected one, which `disable fork` cannot do. The slow job never prints its "finished" message, although the simulation runs beyond time 90.

Exercise 9: A Counter with Assertions and Coverage

Problem. Write a 2-bit counter with enable and asynchronous reset. In the testbench add two concurrent assertions (it increments when enabled, it holds when not enabled) and a covergroup that covers every count value and the wrap-around transition. Report assertion failures and coverage.

fx09_counter_sva_cov.sv

```
module counter2 (
  input  logic       clk, rst_n, en,
  output logic [1:0] count
);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n)  count <= '0;
    else if (en) count <= count + 2'd1;
```

```
endmodule

module fx09_counter_sva_cov;

  logic       clk = 0;
  logic       rst_n, en;
  logic [1:0] count;
  int         fail_cnt;

  counter2 dut (.*);

  always #5 clk = ~clk;

  // ---------------- assertions ----------------
  a_inc: assert property (@(posedge clk) disable iff (!rst_n)
                          en |=> count == $past(count) + 2'd1)
         else begin fail_cnt++; $error("counter did not increment"); end

  a_hold: assert property (@(posedge clk) disable iff (!rst_n)
                           !en |=> count == $past(count))
          else begin fail_cnt++; $error("counter changed while disabled"); end

  // ---------------- coverage ----------------
  covergroup count_cg @(posedge clk iff rst_n);
    cp_value: coverpoint count;                        // four automatic bins
    cp_wrap : coverpoint count { bins wrap = (2'd3 => 2'd0); }
  endgroup

  count_cg cg = new();

  // ---------------- stimulus ----------------
  initial begin
    rst_n = 0;
    en    = 0;
    repeat (2) @(negedge clk);
    rst_n = 1;
    en    = 1;
    repeat (6) @(negedge clk);        // six rising edges with en = 1
    en = 0;
    repeat (2) @(negedge clk);        // two rising edges with en = 0
    $display("count=%0d", count);
    $display("assertion failures=%0d", fail_cnt);
    $display("coverage=%0.2f%%", cg.get_coverage());
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
count=2
assertion failures=0
coverage=100.00%
```

Points to explain.

- The counter is enabled for six clock edges: 0 → 1 → 2 → 3 → 0 → 1 → 2. It then holds 2.
- The assertions use `|=>` because the effect of `en` is visible one clock later, and `$past(count)` is the value at the clock where `en` was sampled. The comparison is done in 2 bits, so 3 + 1 correctly wraps to 0.
- `disable iff (!rst_n)` stops the checks during reset.
- The covergroup samples on each rising clock edge while reset is inactive. The values 0, 1, 2 and 3 are all seen, and so is the transition from 3 to 0, so both coverpoints are fully covered and the covergroup reports 100%.

Exercise 10: A Compact Layered Testbench for the ALU

Problem. Build a complete layered testbench for the course ALU in one file: interface with clocking blocks, transaction, generator, driver, monitor, scoreboard with a reference model, environment, test and top. Send 20 random transactions and report a summary. (M9 develops the same structure in full, one file per class, with line-by-line explanation; this exercise is the version you should be able to write from memory.)

fx10_alu_layered_tb.sv

```
package alu_pkg;
  typedef enum logic [2:0] {
    ALU_ADD, ALU_SUB, ALU_AND, ALU_OR, ALU_XOR, ALU_NOT, ALU_SHL, ALU_SHR
  } alu_op_e;
endpackage

module alu
  import alu_pkg::*;
(
  input  logic       clk, rst_n, valid_in,
  input  logic [7:0] a, b,
  input  alu_op_e    op,
  output logic       valid_out,
  output logic [8:0] result
);
  logic [8:0] result_nxt;

  always_comb begin
    result_nxt = '0;
    case (op)
      ALU_ADD: result_nxt = a + b;
      ALU_SUB: result_nxt = a - b;
      ALU_AND: result_nxt = {1'b0, a & b};
      ALU_OR : result_nxt = {1'b0, a | b};
      ALU_XOR: result_nxt = {1'b0, a ^ b};
      ALU_NOT: result_nxt = {1'b0, ~a};
      ALU_SHL: result_nxt = {a, 1'b0};
      ALU_SHR: result_nxt = {2'b00, a[7:1]};
      default: result_nxt = 'x;
    endcase
  end

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
      valid_out <= 1'b0;
      result    <= '0;
    end
    else begin
      valid_out <= valid_in;
      if (valid_in) result <= result_nxt;
    end
endmodule

interface alu_if (input logic clk);
  import alu_pkg::*;
  logic       rst_n, valid_in, valid_out;
  logic [7:0] a, b;
  alu_op_e    op;
  logic [8:0] result;

  clocking drv_cb @(posedge clk);
    output valid_in, a, b, op;
  endclocking

  clocking mon_cb @(posedge clk);
    input valid_in, a, b, op, valid_out, result;
  endclocking
endinterface

package alu_tb_pkg;
  import alu_pkg::*;
```

```
class alu_txn;
  rand bit [7:0] a, b;
  rand alu_op_e  op;
  logic [8:0]    result;
  int unsigned   id;
endclass

class alu_generator;
  mailbox #(alu_txn) gen2drv_mbx;
  int unsigned       num_txn;

  function new(mailbox #(alu_txn) mbx, int unsigned num_txn);
    this.gen2drv_mbx = mbx;
    this.num_txn     = num_txn;
  endfunction

  task run();
    alu_txn txn;
    for (int unsigned i = 1; i <= num_txn; i++) begin
      txn    = new();                          // one object per transaction
      txn.id = i;
      if (!txn.randomize()) $fatal(1, "randomize failed");
      gen2drv_mbx.put(txn);
    end
  endtask
endclass

class alu_driver;
  mailbox #(alu_txn) gen2drv_mbx;
  virtual alu_if     vif;

  function new(mailbox #(alu_txn) mbx, virtual alu_if vif);
    this.gen2drv_mbx = mbx;
    this.vif         = vif;
  endfunction

  task run();
    alu_txn txn;
    @(vif.drv_cb);
    vif.drv_cb.valid_in <= 1'b0;              // idle value during reset
    wait (vif.rst_n === 1'b1);
    forever begin
      gen2drv_mbx.get(txn);
      @(vif.drv_cb);
      vif.drv_cb.valid_in <= 1'b1;
      vif.drv_cb.a        <= txn.a;
      vif.drv_cb.b        <= txn.b;
      vif.drv_cb.op       <= txn.op;
      @(vif.drv_cb);
      vif.drv_cb.valid_in <= 1'b0;            // one idle cycle between transactions
    end
  endtask
endclass

class alu_monitor;
  mailbox #(alu_txn) mon2scb_mbx;
  virtual alu_if     vif;

  function new(mailbox #(alu_txn) mbx, virtual alu_if vif);
    this.mon2scb_mbx = mbx;
    this.vif         = vif;
  endfunction

  task run();
    alu_txn txn;                              // inputs seen, result still to come
    forever begin
      @(vif.mon_cb);
      if (vif.mon_cb.valid_out === 1'b1 && txn != null) begin
        txn.result = vif.mon_cb.result;       // result of the inputs captured earlier
        mon2scb_mbx.put(txn);
        txn = null;
      end
```

```
      if (vif.mon_cb.valid_in === 1'b1) begin
        txn    = new();
        txn.a  = vif.mon_cb.a;
        txn.b  = vif.mon_cb.b;
        txn.op = vif.mon_cb.op;
      end
    end
  endtask
endclass

class alu_scoreboard;
  mailbox #(alu_txn) mon2scb_mbx;
  int unsigned       pass_cnt, fail_cnt;

  function new(mailbox #(alu_txn) mbx);
    this.mon2scb_mbx = mbx;
  endfunction

  function logic [8:0] predict(bit [7:0] a, bit [7:0] b, alu_op_e op);
    logic [8:0] r;
    case (op)
      ALU_ADD: r = a + b;
      ALU_SUB: r = a - b;
      ALU_AND: r = {1'b0, a & b};
      ALU_OR : r = {1'b0, a | b};
      ALU_XOR: r = {1'b0, a ^ b};
      ALU_NOT: r = {1'b0, ~a};
      ALU_SHL: r = {a, 1'b0};
      ALU_SHR: r = {2'b00, a[7:1]};
      default: r = 'x;
    endcase
    return r;
  endfunction

  task run();
    alu_txn     txn;
    logic [8:0] expected;
    forever begin
      mon2scb_mbx.get(txn);
      expected = predict(txn.a, txn.b, txn.op);
      if (txn.result !== expected) begin
        fail_cnt++;
        $error("a=%0d b=%0d op=%s: result=%0d expected=%0d",
               txn.a, txn.b, txn.op.name(), txn.result, expected);
      end
      else pass_cnt++;
    end
  endtask

  function void report(int unsigned sent);
    $display("scoreboard: sent=%0d checked=%0d passed=%0d failed=%0d",
             sent, pass_cnt + fail_cnt, pass_cnt, fail_cnt);
    if (fail_cnt == 0 && pass_cnt == sent) $display("TEST PASSED");
    else                                   $display("TEST FAILED");
  endfunction
endclass

class alu_env;
  alu_generator      gen;
  alu_driver         drv;
  alu_monitor        mon;
  alu_scoreboard     scb;
  mailbox #(alu_txn) gen2drv_mbx, mon2scb_mbx;
  int unsigned       num_txn;

  function new(virtual alu_if vif, int unsigned num_txn);
    this.num_txn = num_txn;
    gen2drv_mbx  = new();
    mon2scb_mbx  = new();
    gen = new(gen2drv_mbx, num_txn);
    drv = new(gen2drv_mbx, vif);
    mon = new(mon2scb_mbx, vif);
    scb = new(mon2scb_mbx);
```

```
    endfunction

    task run();
      fork
        gen.run();
        drv.run();
        mon.run();
        scb.run();
      join_none
      fork
        wait (scb.pass_cnt + scb.fail_cnt == num_txn);
        begin
          #100000;
          $error("timeout: the scoreboard did not receive all transactions");
        end
      join_any
      disable fork;                          // end of test: stop all components
      scb.report(num_txn);
    endtask
  endclass

  class alu_test;
    alu_env env;

    function new(virtual alu_if vif);
      env = new(vif, 20);
    endfunction

    task run();
      env.run();
    endtask
  endclass

endpackage

module fx10_alu_layered_tb;
  import alu_tb_pkg::*;

  logic clk = 0;
  always #5 clk = ~clk;

  alu_if vif (clk);

  alu dut (
    .clk       (clk),
    .rst_n     (vif.rst_n),
    .valid_in  (vif.valid_in),
    .a         (vif.a),
    .b         (vif.b),
    .op        (vif.op),
    .valid_out (vif.valid_out),
    .result    (vif.result)
  );

  alu_test test;

  initial begin
    vif.rst_n = 0;
    repeat (2) @(negedge clk);
    vif.rst_n = 1;
  end

  initial begin
    test = new(vif);
    test.run();
    $finish;
  end
endmodule
```

EXPECTED OUTPUT

```
scoreboard: sent=20 checked=20 passed=20 failed=0
TEST PASSED
```

How the pieces fit.

Step	What happens
Top	Creates the clock, the interface instance and the DUT; applies reset for two clocks; constructs the test with the interface and runs it.
Environment constructor	Creates the two mailboxes once and hands the same handle to both ends of each connection; passes the virtual interface to the driver and monitor.
Generator	Creates a new object for every transaction, randomizes it, checks the result, and puts it in `gen2drv_mbx`. It needs no simulation time.
Driver	Waits for reset to end. For each transaction: waits for a clock, drives the inputs with `valid_in` = 1 through `drv_cb`, and one clock later sets `valid_in` back to 0.
DUT	Samples the inputs at the next rising edge; `result` and `valid_out` are visible after that edge.
Monitor	On every clock, through `mon_cb`: if `valid_out` is 1, it completes the pending transaction with `result` and sends it; if `valid_in` is 1, it starts a new transaction with the sampled inputs. The result check comes first so that back-to-back transactions would also be paired correctly.
Scoreboard	Computes the expected 9-bit result from the observed inputs and compares with `!==`.
End of test	The environment waits until the scoreboard has checked all 20 transactions (or a timeout), stops the components and prints the report.

Points an interviewer will probe.

- Why a new `alu_txn` per loop iteration in the generator and the monitor (Q172, Q183).
- Why the driver and monitor use clocking blocks instead of the raw clock (Q113).
- Why the monitor pairs inputs with the result one clock later (Q184).
- Why the summary line checks `pass_cnt == sent`, not only `fail_cnt == 0` (Q305).
- What `disable fork` kills here and why that is acceptable at the end of the test (Q135).
- How a test would restrict the stimulus to additions only without editing the generator (Q254).